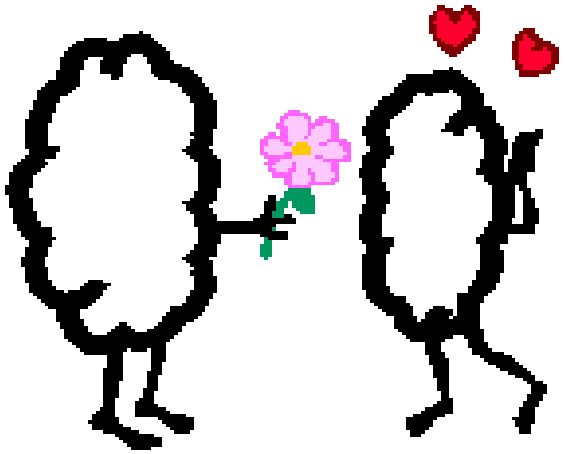
Chapter 4

Relationships Between Objects



Contents



- Associations and Links
- Multiplicity
- Aggregation
- Collection of Objects
- Inheritance
- Overriding
- “super” Keyword
- equals() and toString() method

4.1 Associations and Links



Associations

- The formal name for a structural relationship that exists between classes is an **association**.
- Some sample associations:
  - A Professor ***teaches*** many Students.



  - A Student ***is enrolled in*** a Course.

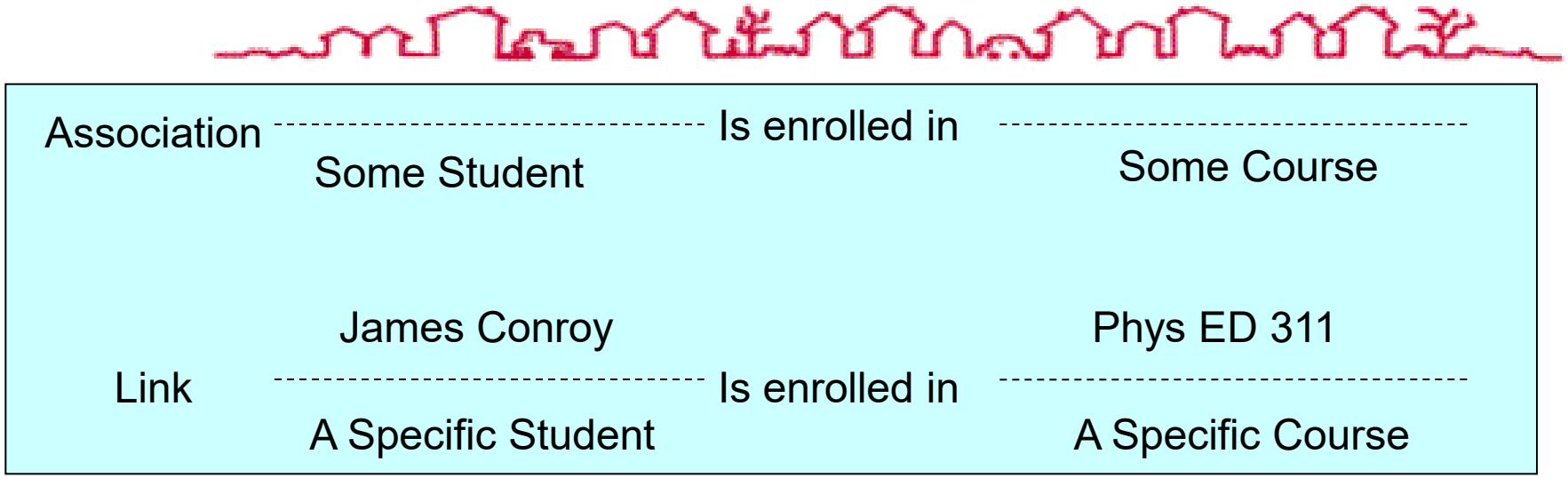


Links



- **link** is used to refer to a structural relationship that exists between two specific ***objects*** (***instances***).
- Given the association "a Student *is enrolled in* a Course," we might have the following links:
  - Chloe Shylow (a particular Student object) is enrolled in Math 101 (a particular Course object).
  - Fred Schnurd (a particular Student object) is enrolled in Basketweaving 972 (a particular Course object).
  - Mary Smith (a particular Student object) is enrolled in Basketweaving 972 (a particular Course object; as it turns out, the ***same*** Course object that Fred Schnurd is linked to).

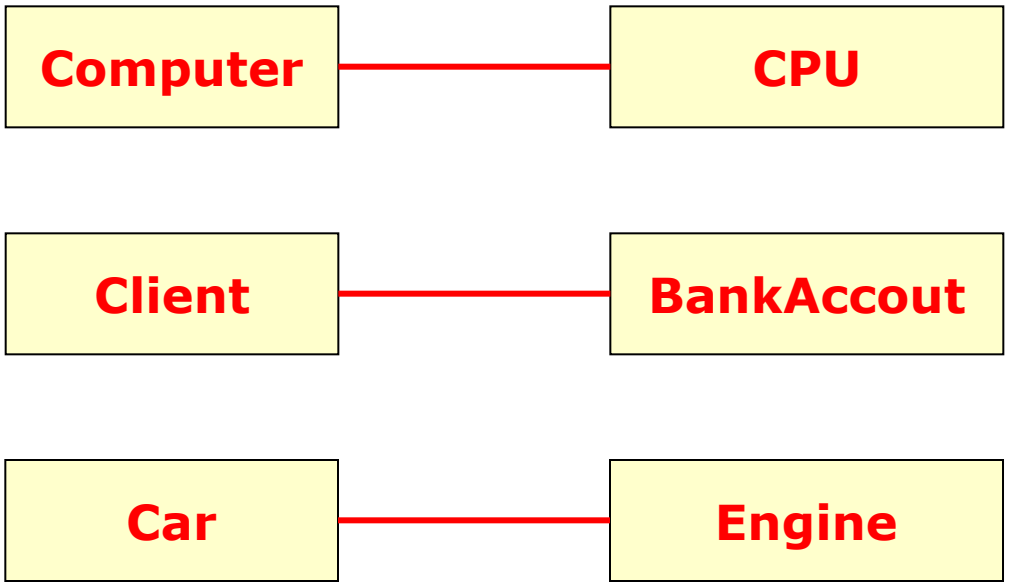
Associations and Links



- Yet another way to think of the difference between an association and a link is that
  - An association is a ***potential*** relationship between objects of a certain type/class.
  - A link is an ***actual*** relationship between objects of those particular types.

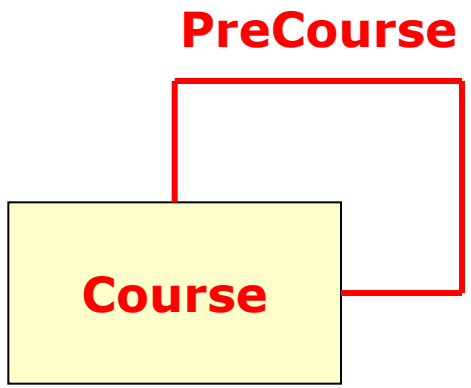
Binary Associations

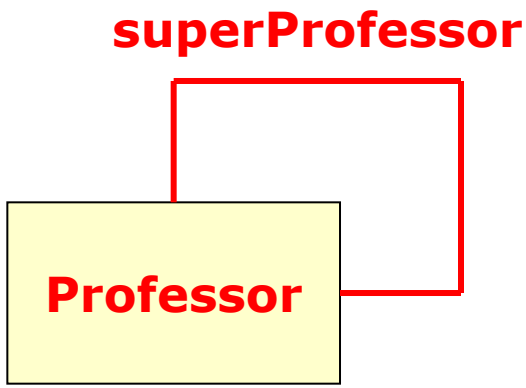
- Most associations arise between two ***different*** classes; such associations are known as ***Binary Associations*.**



Unary or Reflexive association

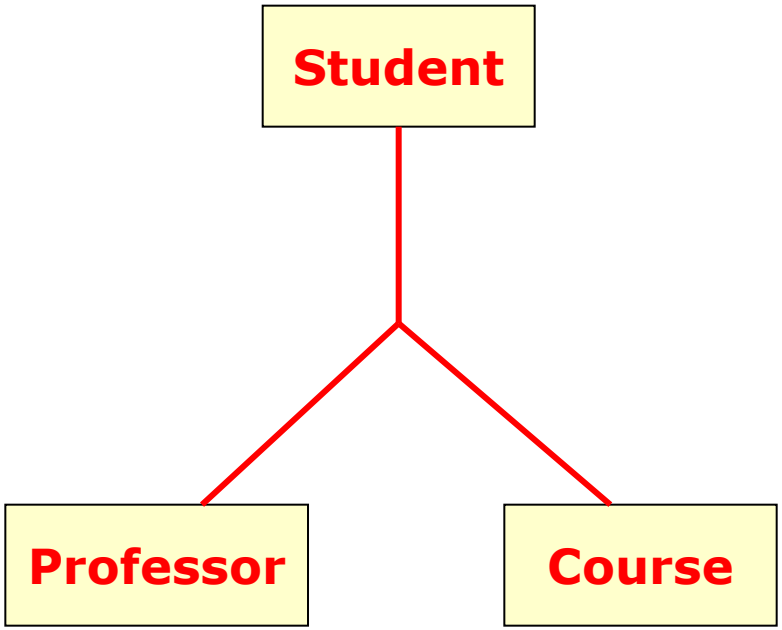
- A **unary**, or **reflexive**, **association** is between two instances of the ***same class***, for example:
  - A Course ***is a prerequisite for*** (another) Course(s).
  - A Professor ***supervises*** (other) Professor(s).





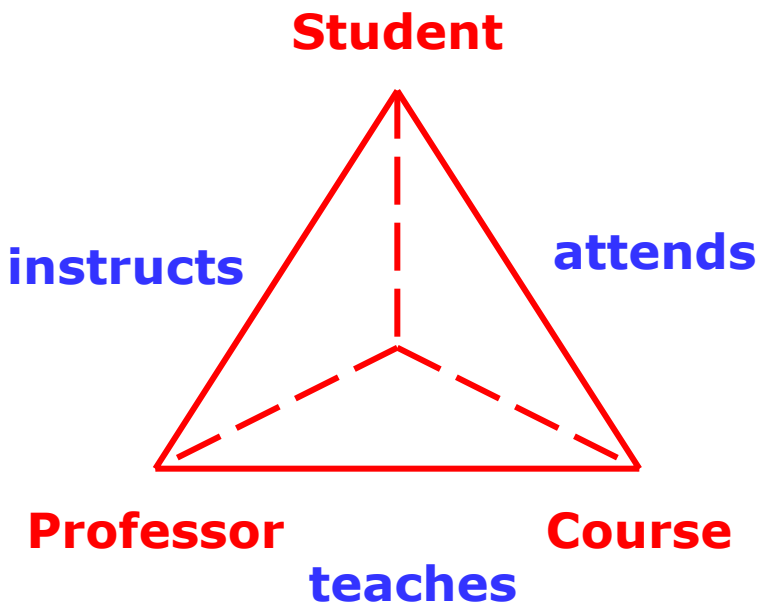
Ternary Association

- A **ternary association** involves three classes. for example,
  - "a Student takes a Course from a particular Professor".



Ternary Association

- We typically decompose higher-order associations into an appropriate number of binary associations.We can, for example, represent the preceding three-way association as three binary associations instead :
  - A Student ***attends*** a Course.
  - A Professor ***teaches*** a Course.
  - A Professor ***instructs*** a Student.



4.2 Multiplicity

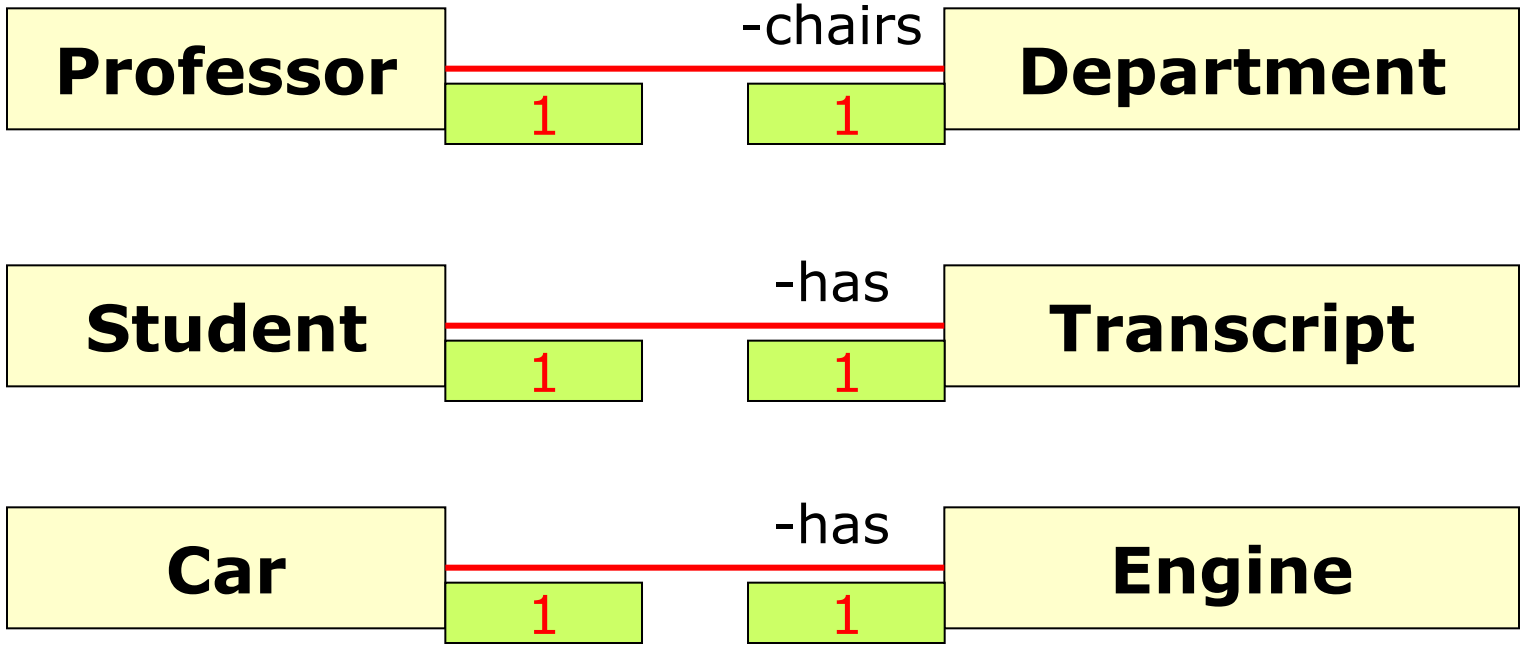


Multiplicity

- For a given association type X between classes A and B, the term **multiplicity** refers to the number of objects of type A that may be associated with a given instance of type B.
  - For example, a Student attends ***multiple*** Courses, but a Student has only ***one*** Professor in the role of advisor.
- There are three basic "flavors" of multiplicity:
  - **one-to-one**
  - **one-to-many**
  - **many-to-many**

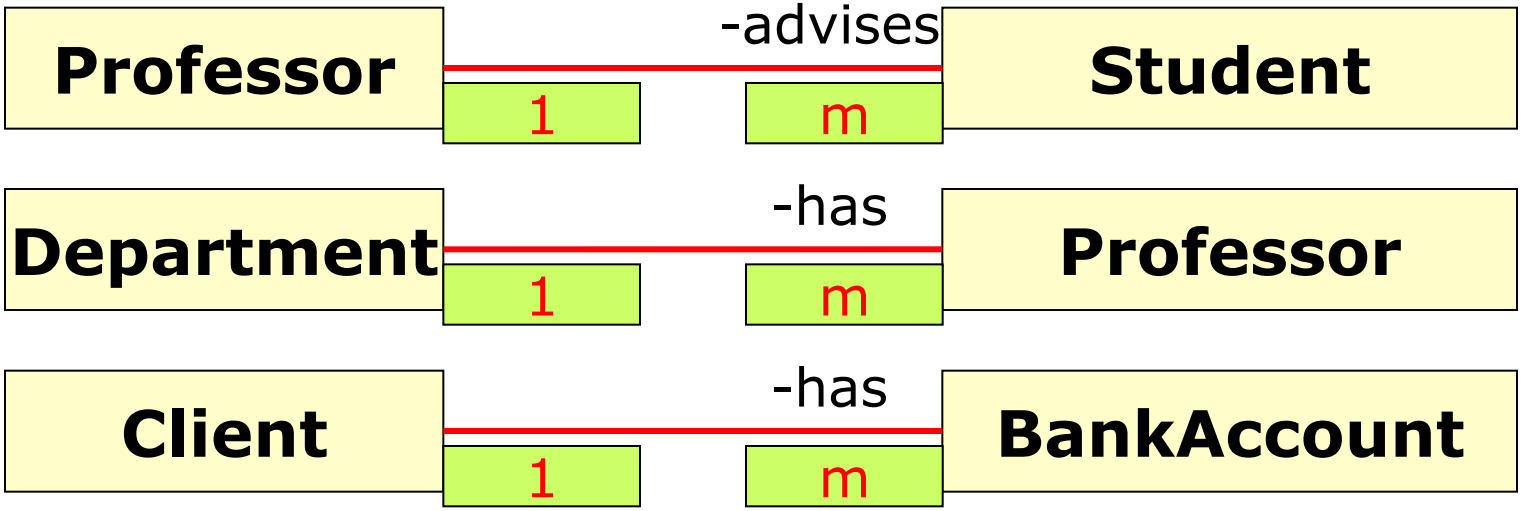
one-to-one(1:1)

- With **a one-to-one (1:1)** association, exactly one instance of class A is related to exactly one instance of class B—no fewer, no more, and vice versa.



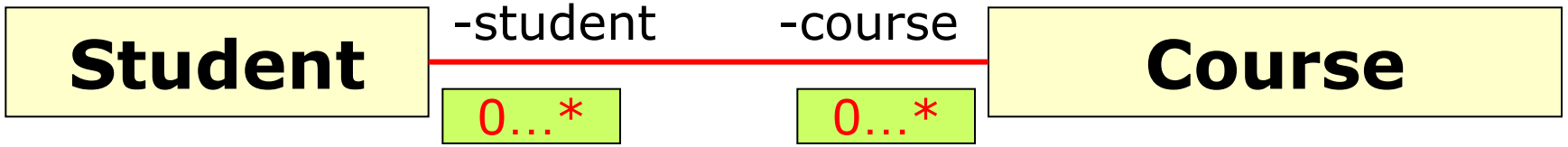
one-to-many(1:m)

- In a **one-to-many (1:m)** association, there can be ***many*** instances of class B related to a ***single*** instance of class A in a particular fashion; but, from the perspective of an instance of class A that is so related.



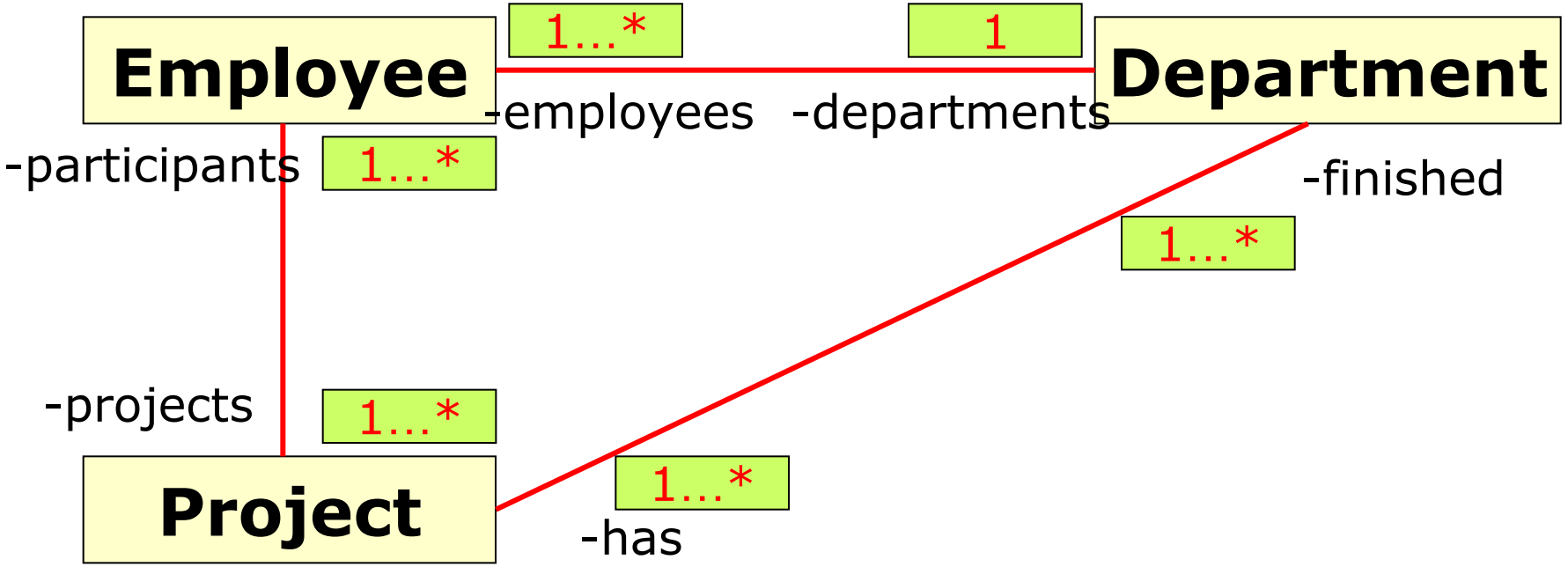
many-to-many(m:m)

- With **a many-to-many (m:m)** association, a given single instance of class A can have many instances of class B related to it, and vice versa.



- "many" can be interpreted as ***zero*** or more (***optional***) or as ***one*** or more (***mandatory***) at either end of an (m:m) association.

many-to-many(m:m)

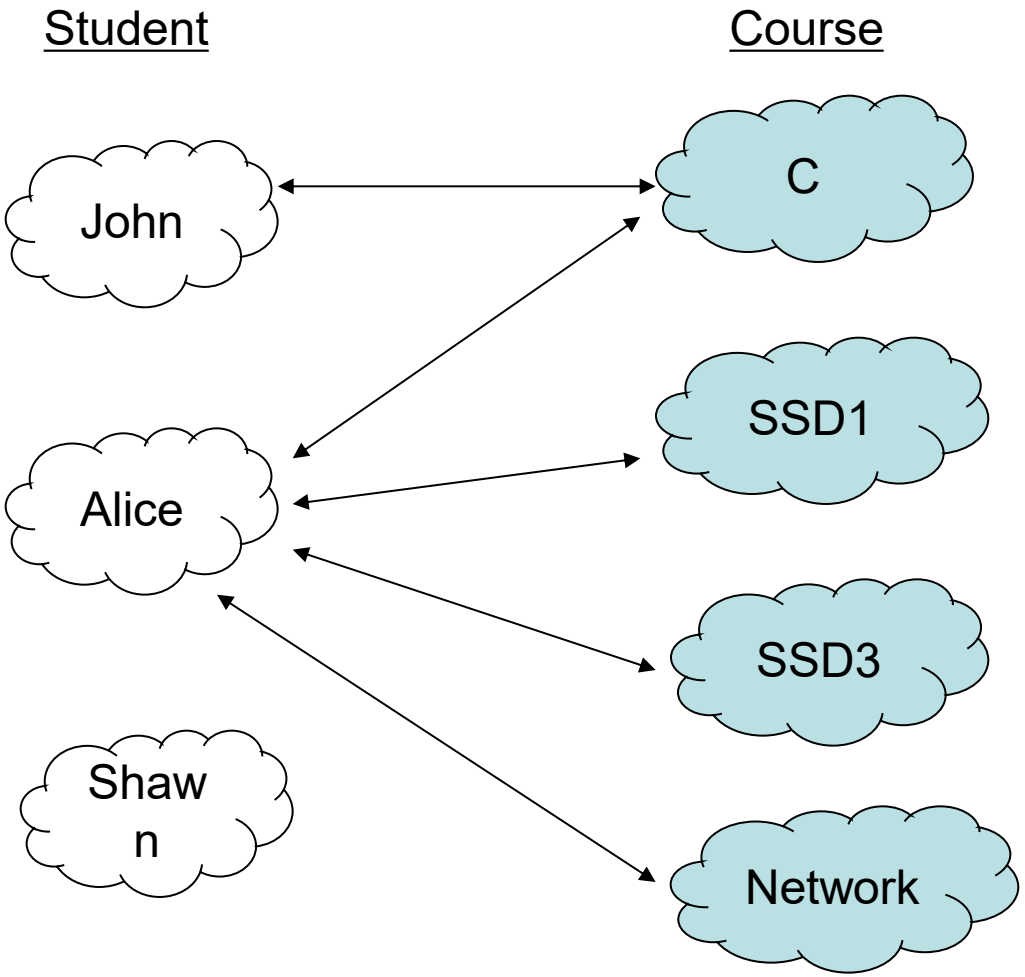


Multiplicity and Links

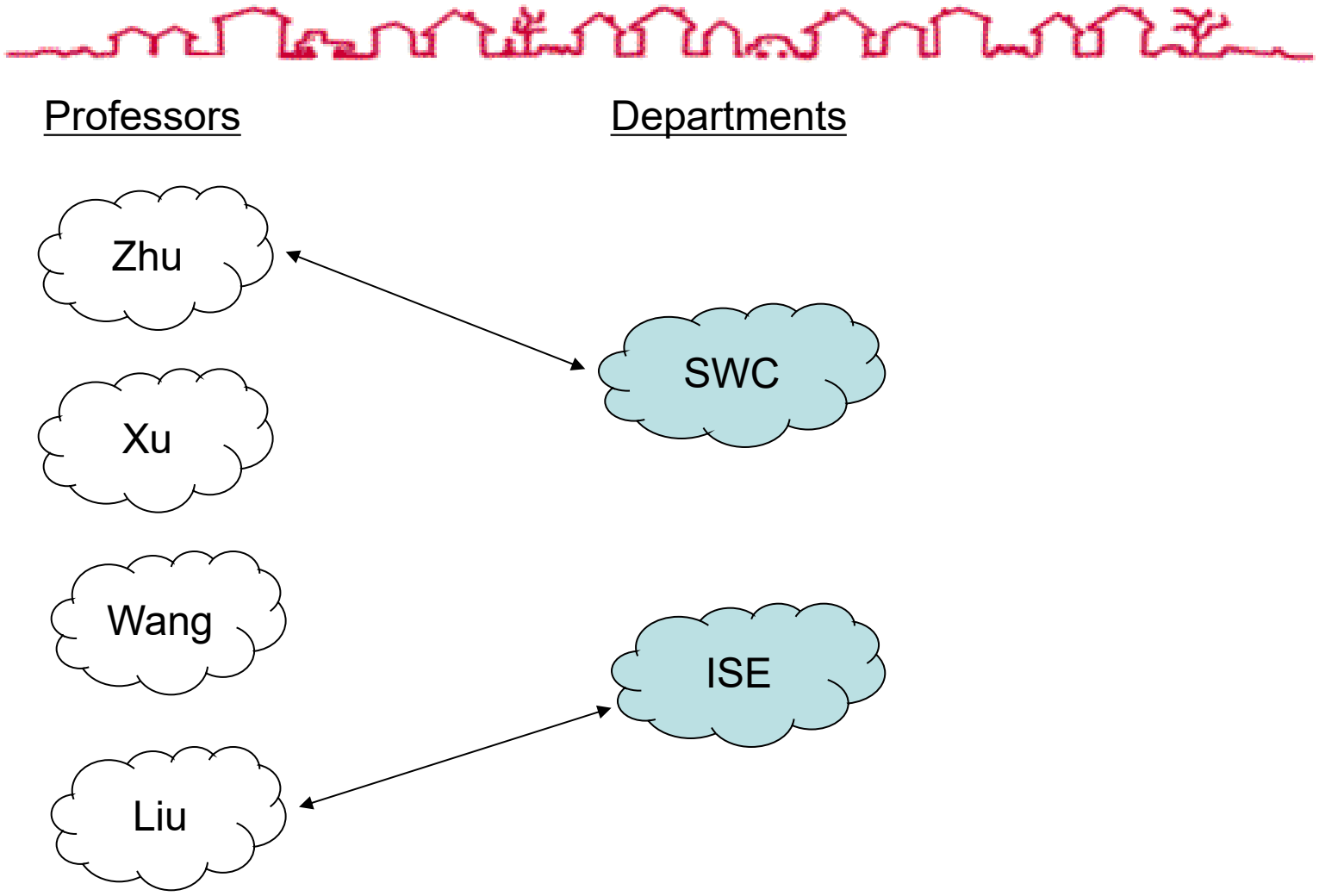


- Note that the concept of **multiplicity** pertains to **associations**, but not to **links**. ***Links always exist in pairwise fashion between two objects***. Therefore, multiplicity in essence defines how many links of a certain association type can originate from a given object.

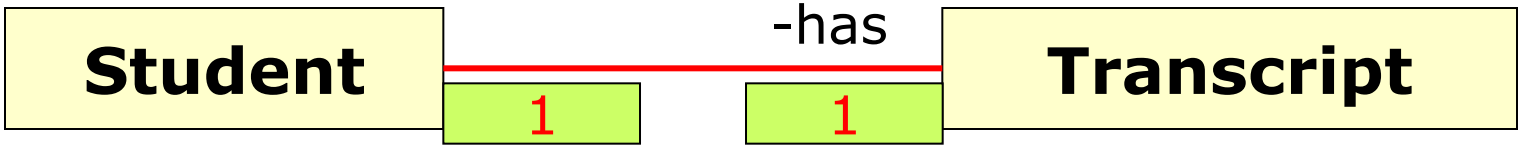
Multiplicity and Links



Multiplicity and Links

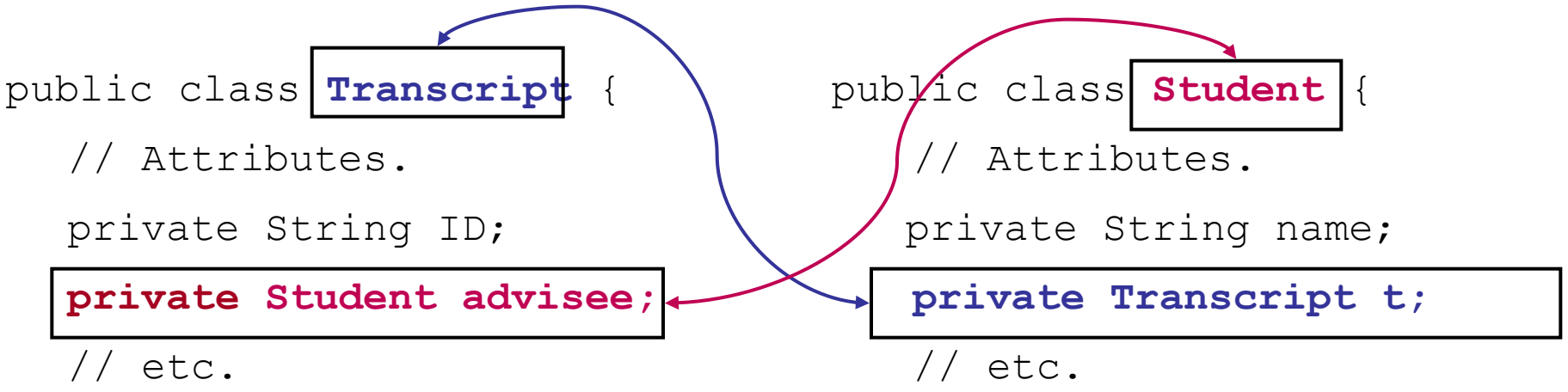


Representing Associations In Code -- An Example



Representing Associations In Code -- An Example

- A Transcript belongs to one Student, and a Student has exactly one Transcript as an advisor



  - We've seen this before! Each class holds a reference to the other as an attribute

Associations in Code, cont.

- The get/set methods look like this:

```
public class Student {
    private Transcript t;
    // etc.
    public void setTranscript(Transcript a) {
            t = a;
    }
    public Transcript getTranscript() {
            return t;
    }
```

```
public class Transcript {
    private Student student;
    // etc.
    public void setStudent(Student a) {
            student = a;
    }
    public Student getStudent() {
            return student;
    }
```

Associations in Code, cont.



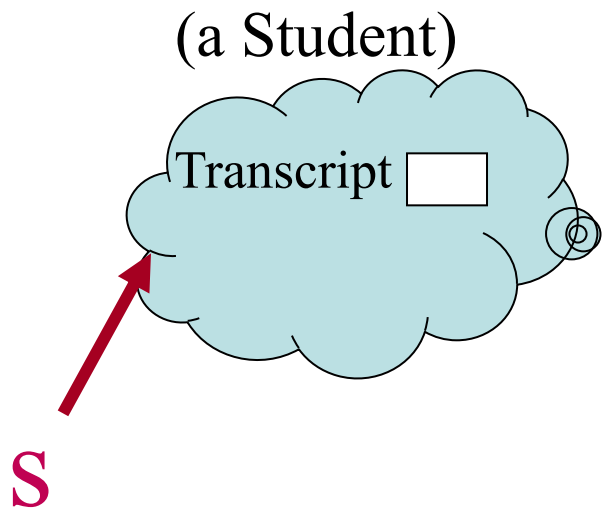
- The client code needed to bidirectionally link objects at run time would thus be as follows:

```
public static void main(String[] args) {
    Student s = new Student();
    Transcript p = new Transcript();
    // details omitted ...
    // Call the reciprocal "set" methods.
    s.setTranscript(p);
    p.setStudent(s);
}
```

Associations in Code, cont.

- Memory Schematic:

```
Student s = new Student();
```

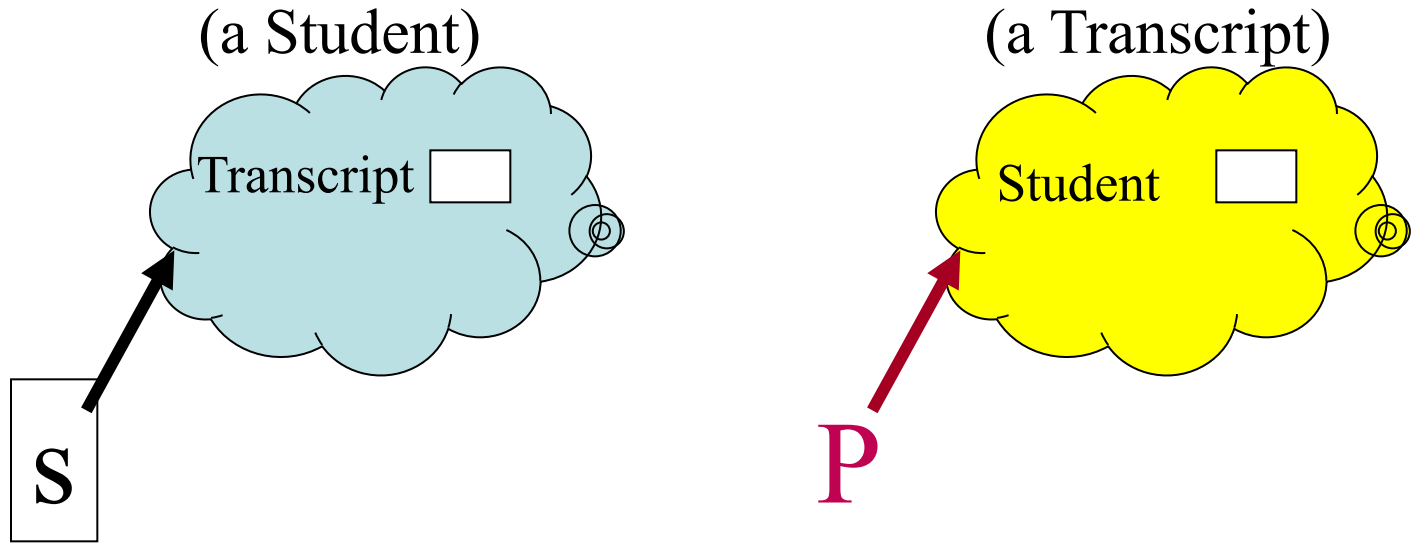


Associations in Code, cont.

- Memory Schematic:

```
Student s = new Student();

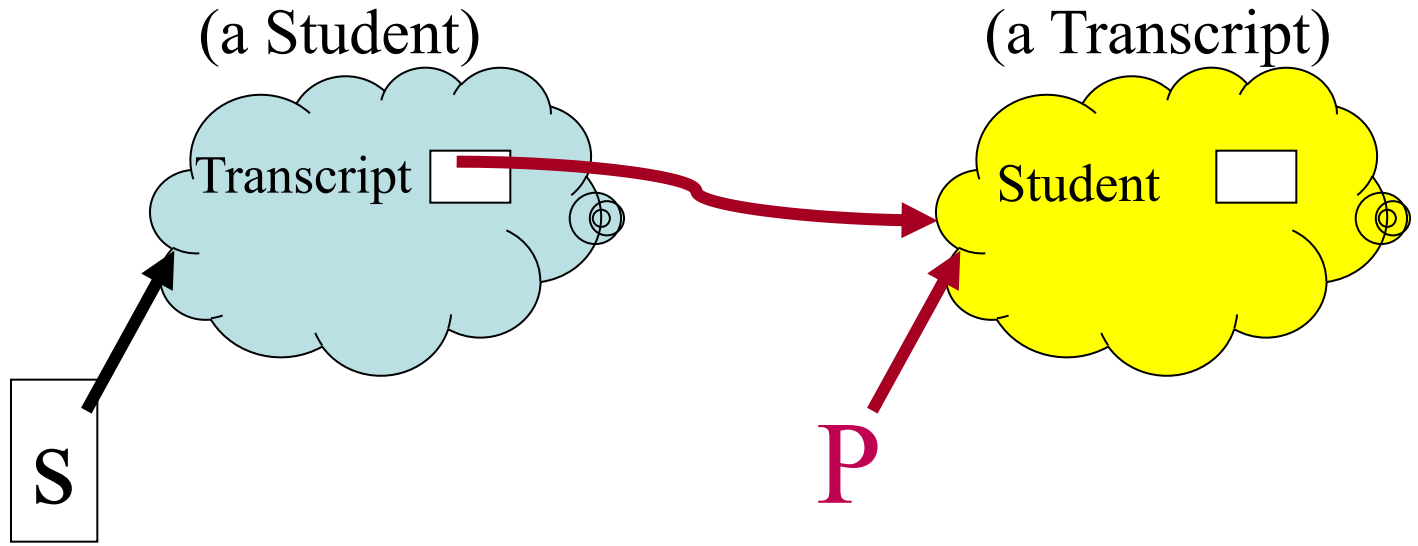
Transcript p = new Transcript();
```



Associations in Code, cont.

- Memory Schematic:

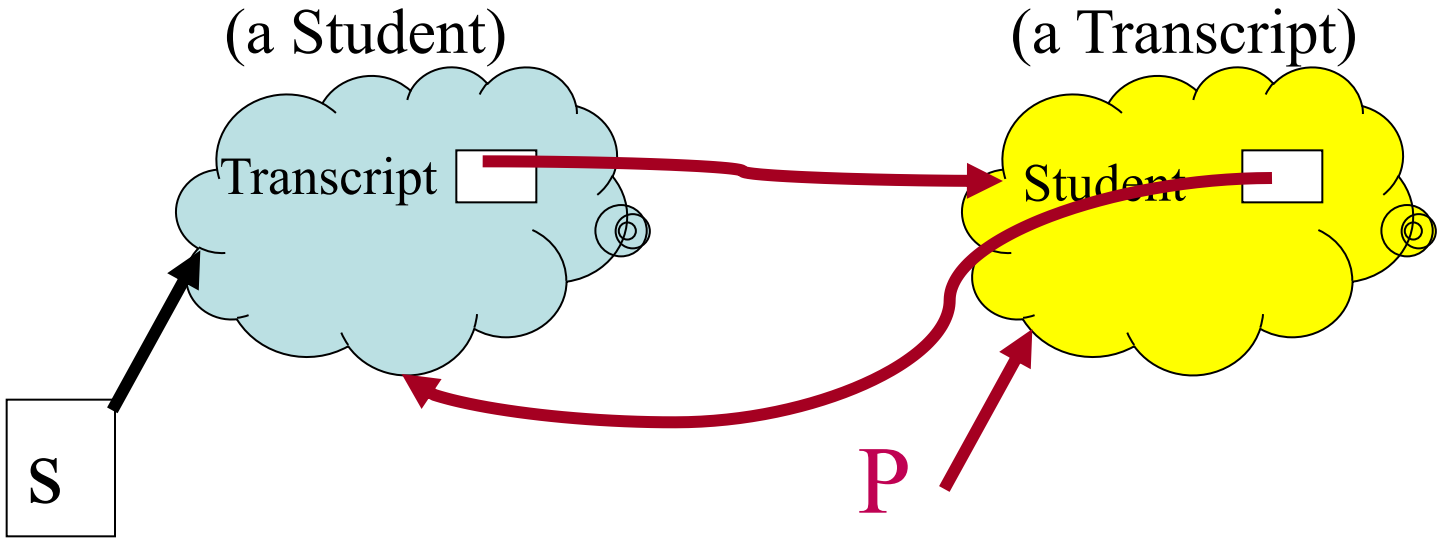
```
Student s = new Student();
Transcript p = new Transcript();
s.setTranscript(p);
```



Associations in Code, cont.

- Memory Schematic:

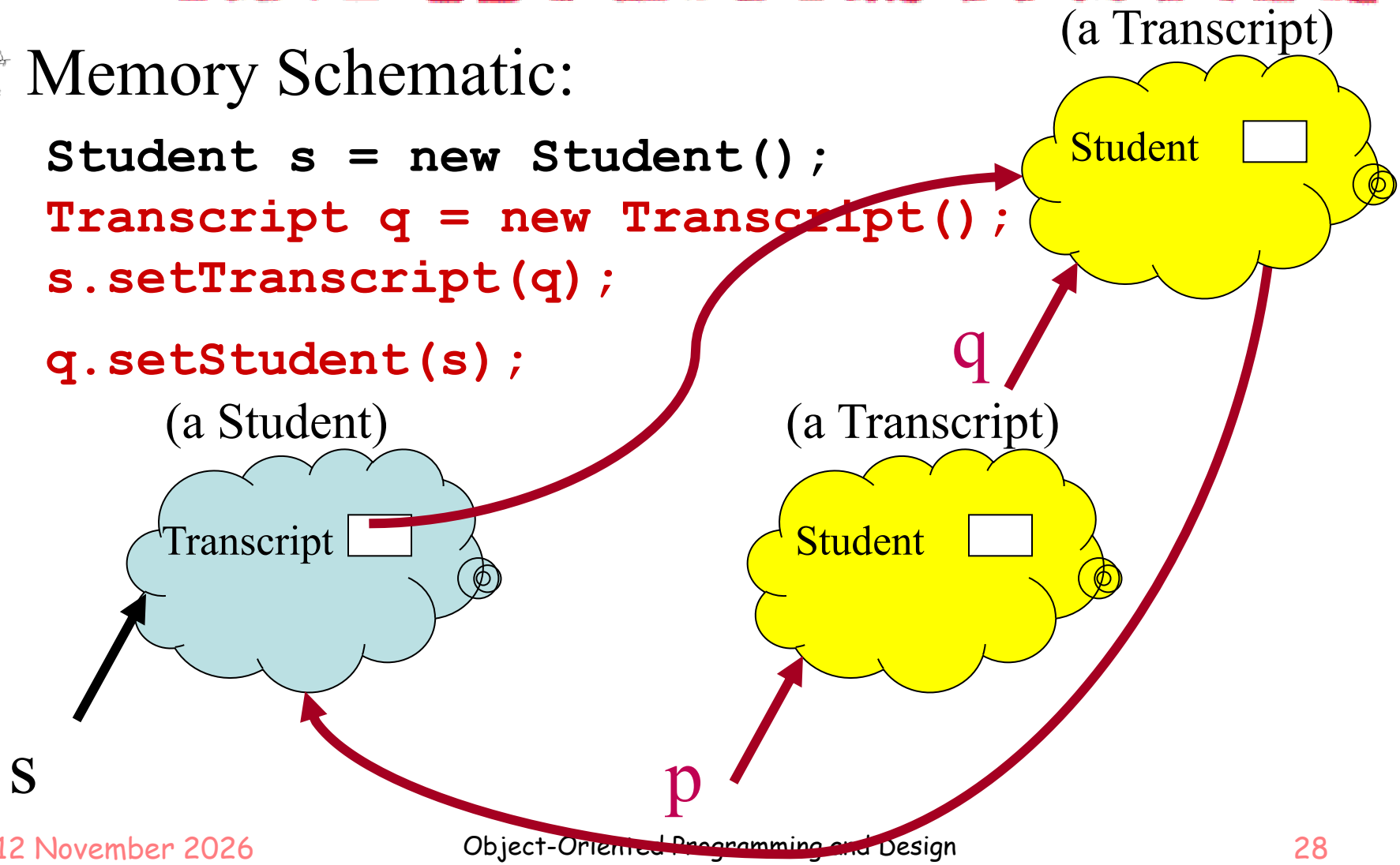
```
Student s = new Student();
Transcript p = new Transcript();
s.setTranscript(p);
p.setStudent(s);
```



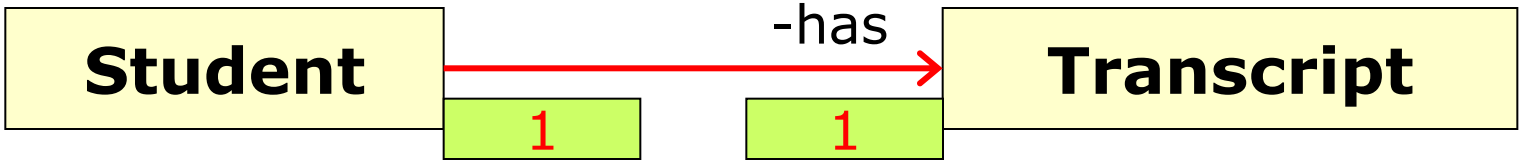
Associations in Code, cont.

- Memory Schematic:

```
Student s = new Student();
Transcript q = new Transcript();
s.setTranscript(q);
q.setStudent(s);
```



Eg1:




```
public class Transcript {
   private String courseName;
    private double grade;

    //…
}
```

```
public class Student {
    private String degree;
    private String major;
    private Transcript t;

    public String getDegree() {
         return this.degree;
    }

    public String getMajor() {
         return this.major;
    }

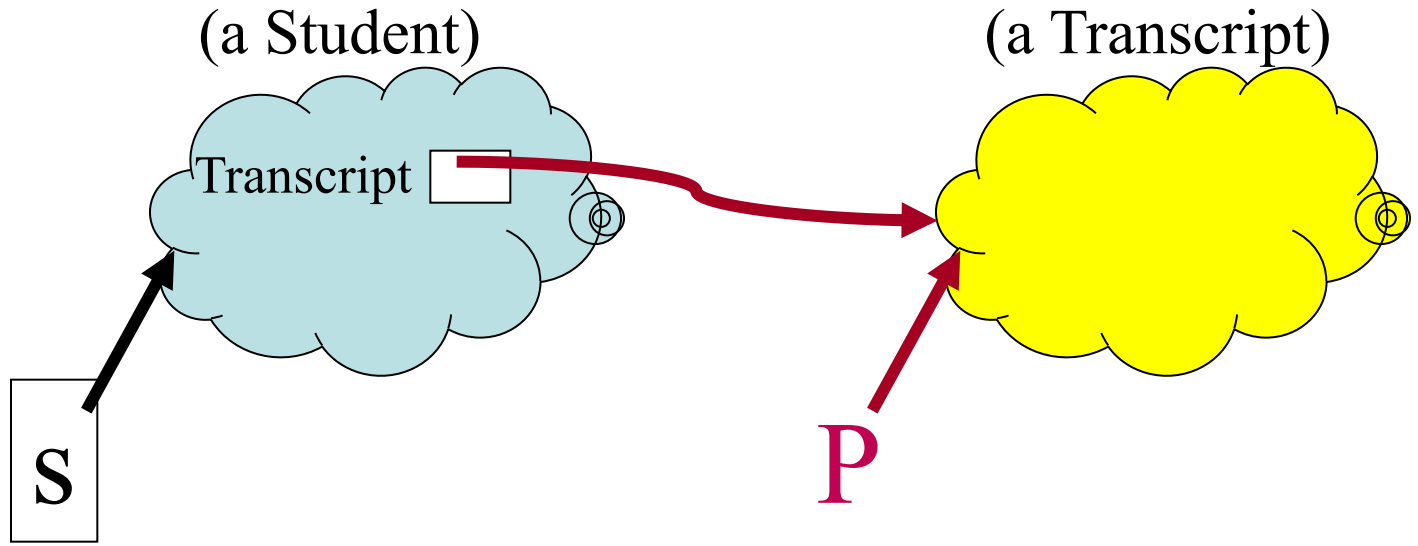
    public Transcript getT(){
         return this.t;
    }
}
```

-o

Associations in Code, cont.

- Memory Schematic:

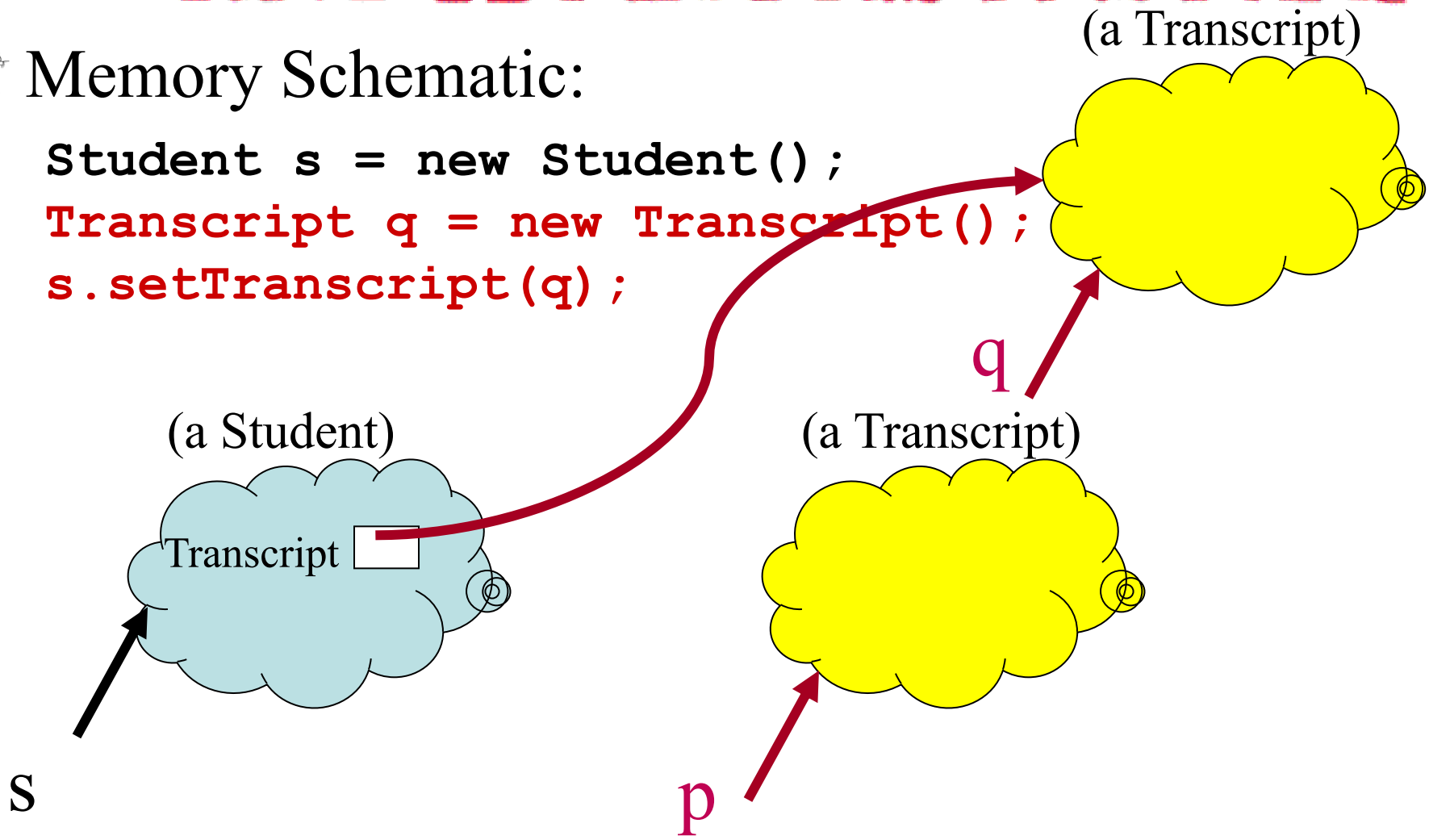
```
Student s = new Student();
Transcript p = new Transcript();
s.setTranscript(p);
```



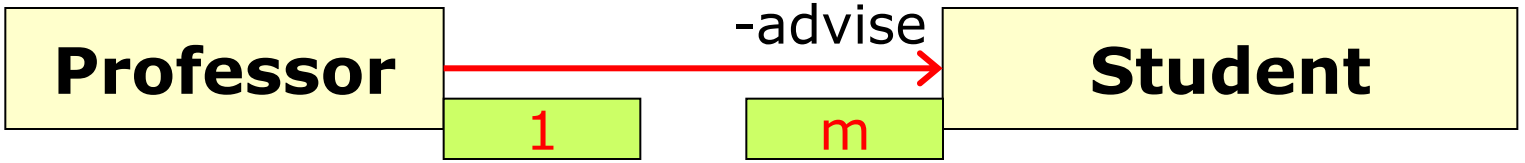
Associations in Code, cont.

- Memory Schematic:

```
Student s = new Student();
Transcript q = new Transcript();
s.setTranscript(q);
```



Eg2:




```
public class Student {
   private String degree;
   private String major;
   private Transcript transcript;

   public String getDegree() {
        return this.degree;
   }

   public String getMajor() {
        return this.major;
   }

   public Transcript
   getTranscript(){
        return this.transcript;
   }
}
```

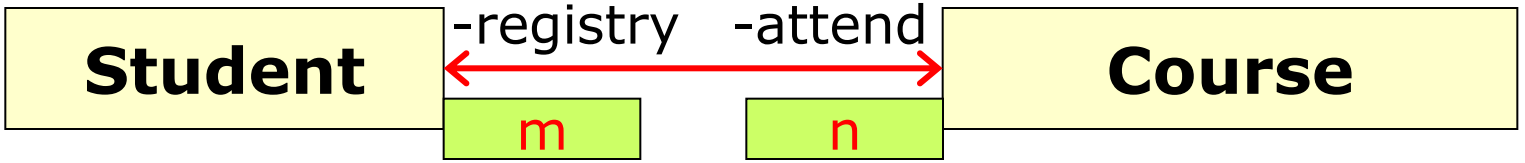
```
public class Professor {
   private String ID;
   private String name;
   private Student advise[];

   public String getID() {
        return this.ID;
   }

   public String getName() {
        return this.name;
   }

   //… …
}
```

Eg3:




```
public class Student {
   private String degree;
   private String major;
   private Course course[];

   public String getDegree() {
        return this.degree;
   }

   public String getMajor() {
        return this.major;
   }


}
```

```
public class Course {
   private String courseNO;
   private String name;
   private double credit;
   private Student student[];

   public String getCourseID() {
        return this.courseID;
   }

   public String getName() {
        return this.name;
   }

   public String getCredit() {
        return this.credit;
   }
}
```

Eg3:




```
public class Student {
   private String degree;
   private String major;


   public String getDegree() {
        return this.degree;
   }

   public String getMajor() {
        return this.major;
   }


}
```

```
public class Course {
   private String courseNO;
   private String name;
   private double credit;
   private Student student[];

   public String getCourseID() {
        return this.courseID;
   }

   public String getName() {
        return this.name;
   }

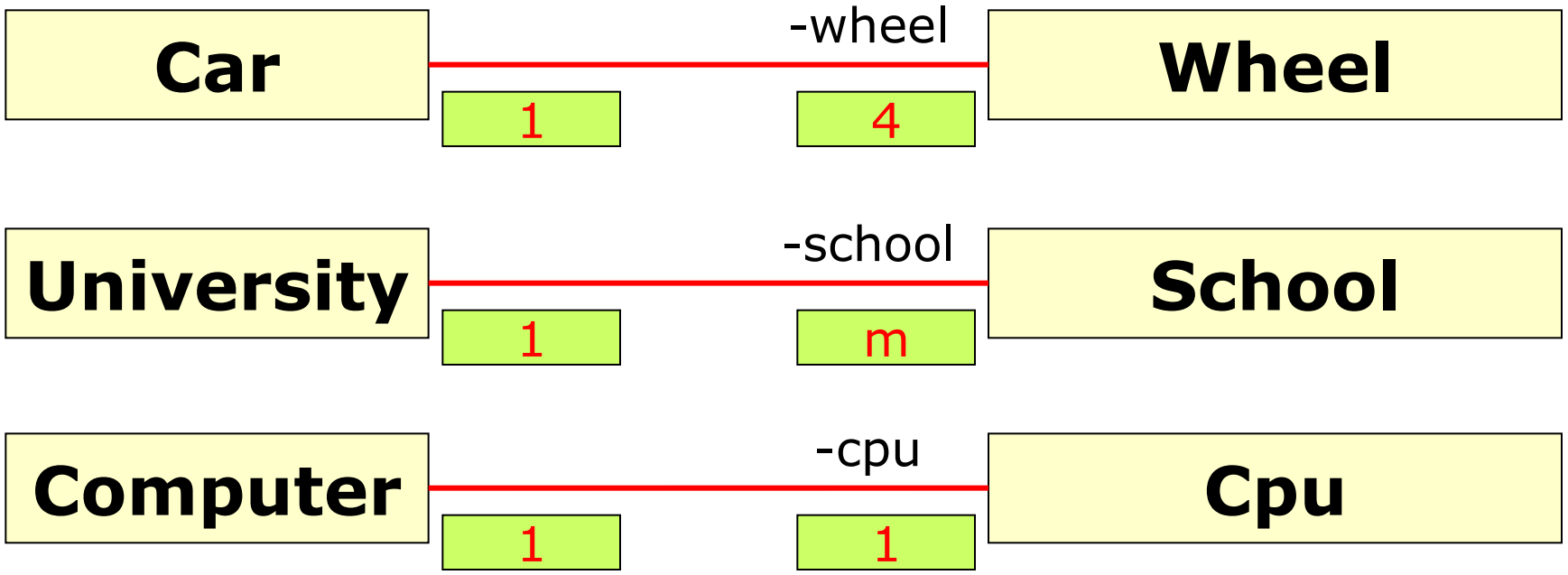
   public String getCredit() {
        return this.credit;
   }
}
```

4.3 Aggregation and Composition



Aggregation

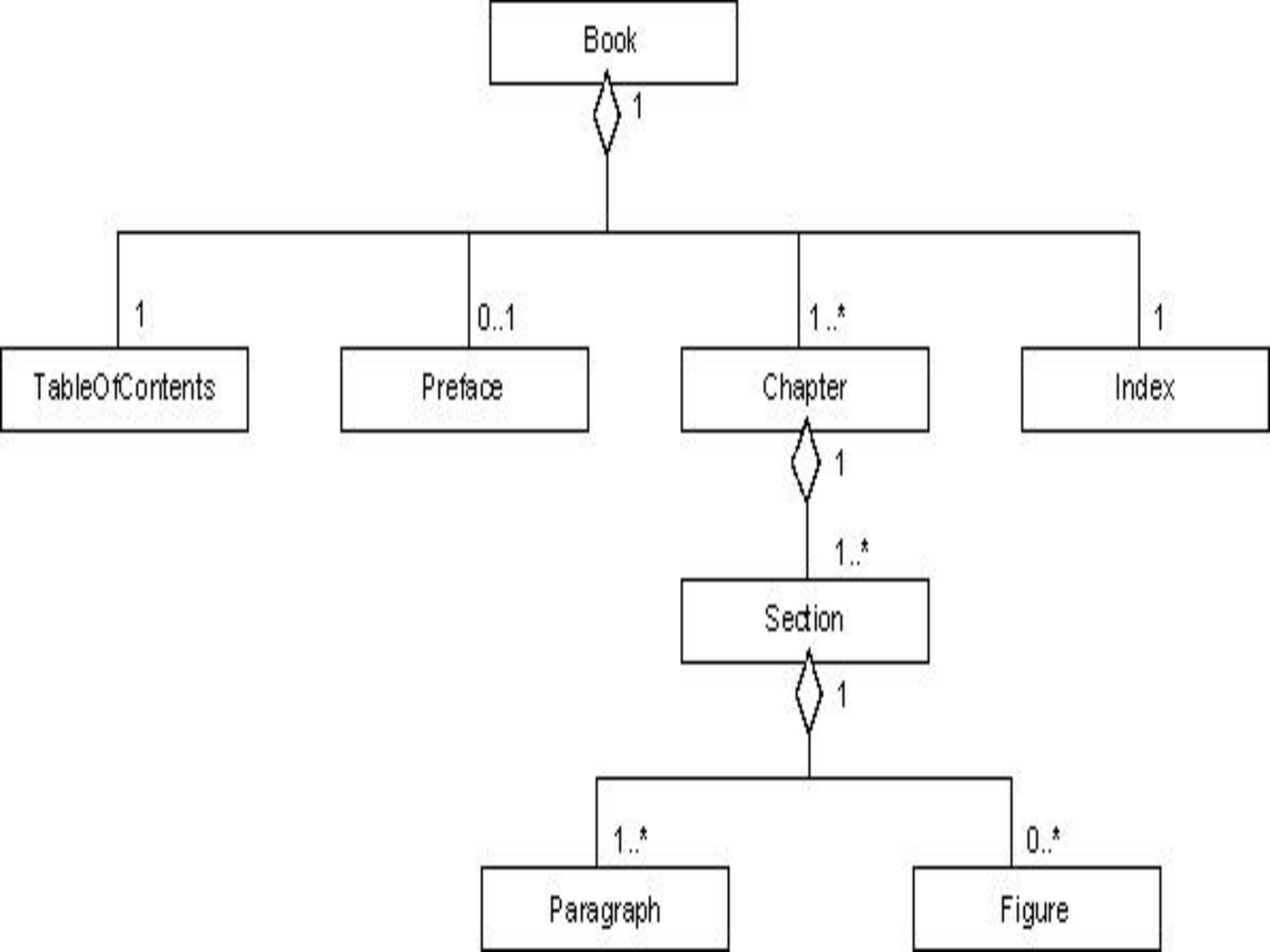
- **Aggregation** is a special form of association, alternatively referred to as the “consists of”, “is composed of”, or “has a” relationship.



Composition



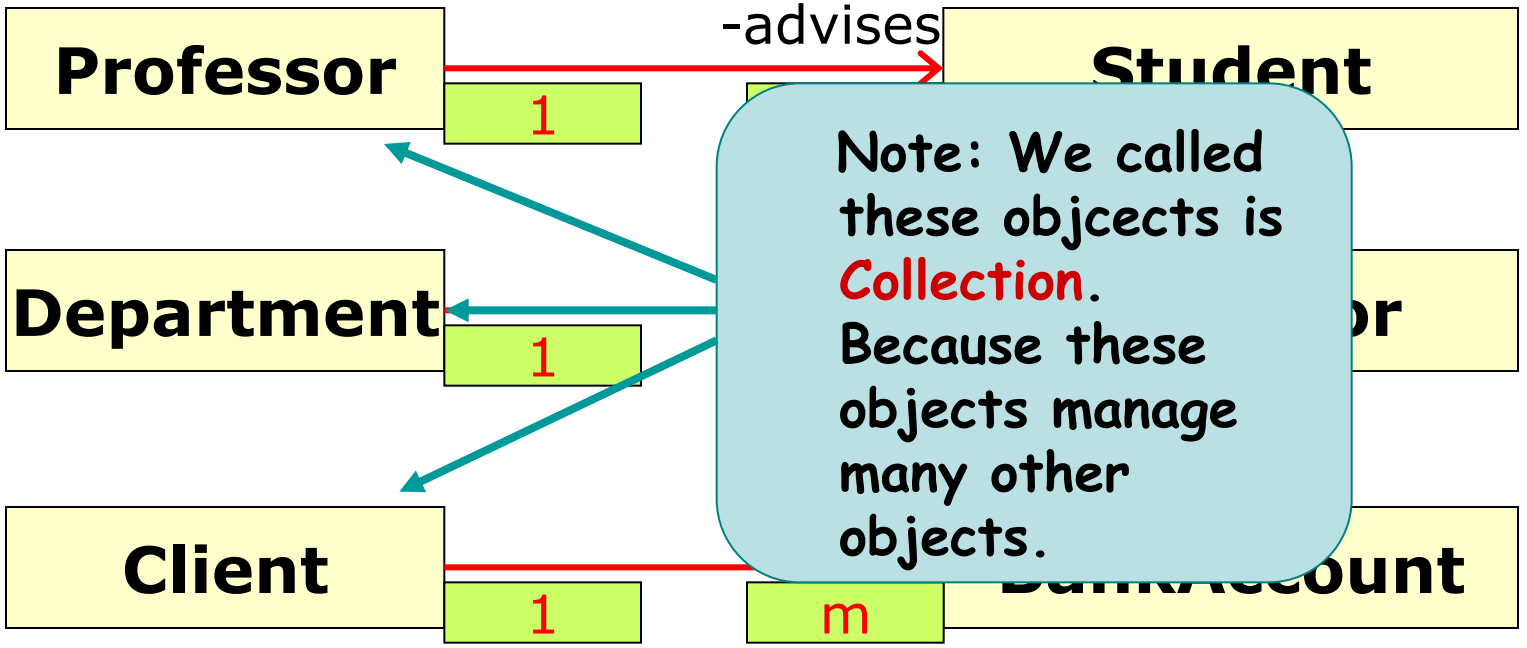
- ***Composition*** is a strong form of aggregation, in which the "parts" cannot exist without the "whole."
- For example, a book is composed of one table of contents, zero or one prefaces, one or more chapters, and one index. In a book, each chapter is composed of one or more sections, and each section is composed of one or more paragraphs and zero or more figures.


Book
1
1
0..1
1..*
1
TableOfContents
Preface
Chapter
Index
1
1..*
Section
1
1..*
0..*
Paragraph
Figure

4.4 Collection of Objects



Collections



Java Collection Class

- The Java language predefines a number of different collection classes.
  - Array
  - ArrayList
  - Vector
  - HashMap
  - TreeMap

Instantiate

```
public class Student {
   private String name;
   private String ID;
   private ArrayList<Course> course;

   public Student(String initialName, String initialID){
        name = initialName;
        ID = initialID;
        course = new ArrayList<Course>();
   }

   public String getName() {
        return this.name;
   }

   public String getID() {
        return this.ID;
   }
}
```

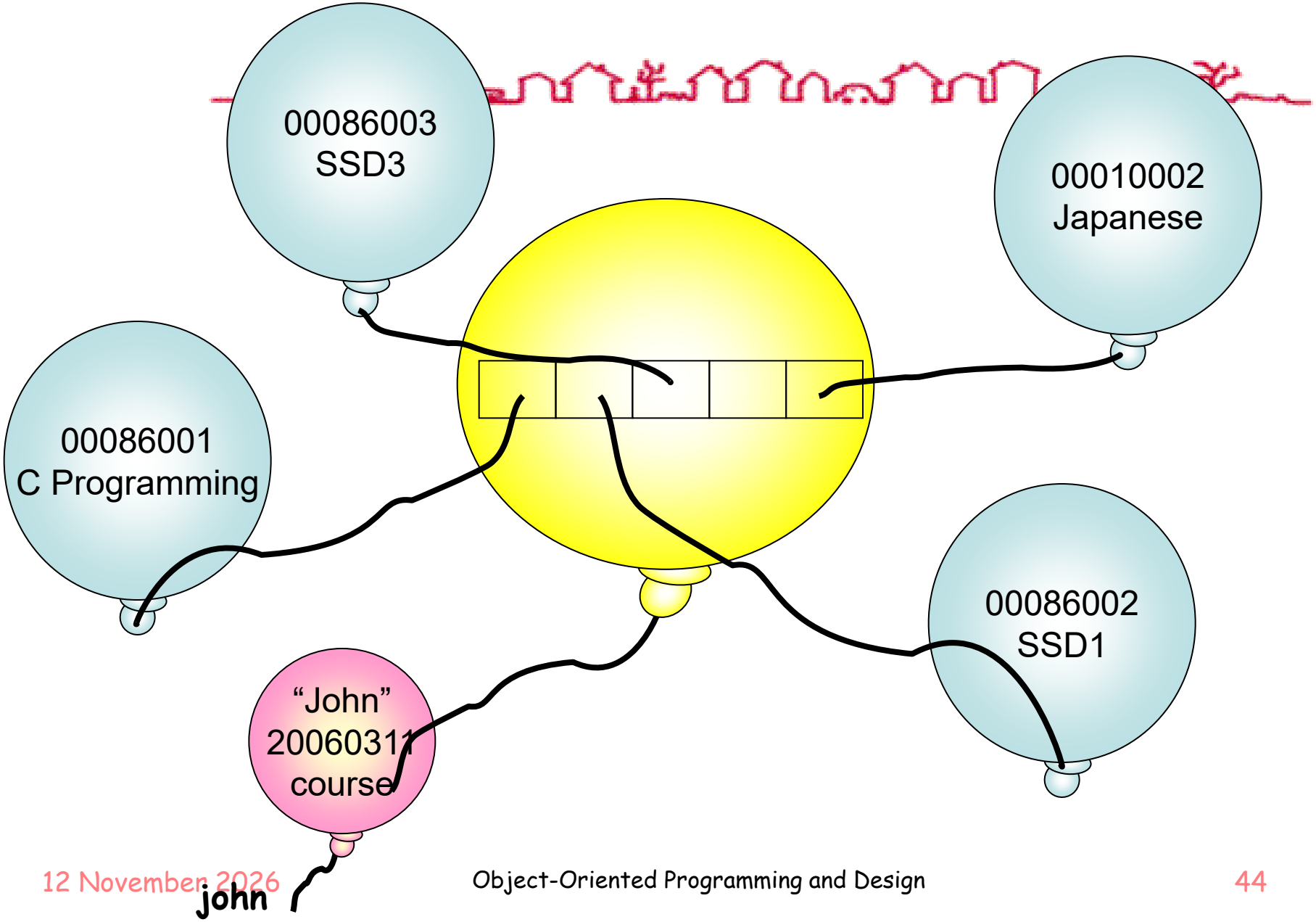
Collections Organize References to Other Objects





Collection stores ***references*** to such objects. That is, the objects being organized by a collection live physically ***outside*** of the collection in the JVM's memory; only their ***handles*** reside inside of the collection.

Collections Organize References to Other Objects



Collections Organize References to Other Objects



- So collections, regardless of type and regardless of the programming language in which they are implemented, provide, at a minimum, methods for
  - Adding objects
  - Removing objects
  - Retrieving specific individual objects
  - Iterating through the objects in some predetermined order
  - Getting a count of the number of objects presently referenced by the collection
  - Answering a true/false question as to whether a particular object's reference is in the collection or not

Array



- An array is a simple type of ordered list.
- Group data objects of the same type
- Declare arrays of primitive or class types

```
char s[];
Point p[];
char [] s;
Point [] p;
```

- Create space for a reference
- Remember an array is an object not memory reserved for primitive types

Creating Array



- Use the **new** keyword to create an array object.

```
s = new char[20];
p = new Point[100];
p[0] = new Point();
p[1] = new Point();
```

Initializing Arrays

```
1 String names[];
2 names = new String[3];
3 names[0] = "Georgianna";
4 names[1] = "Jen";
5 names[2] = "Simon";
6
7 Myclass array[] = {
8 new Myclass(),
9 new Myclass(),
10 new Myclass()
11 };
12
13 Color palette[] = {
14 Color.blue,
15 Color.red,
16 Color.white
17 };
```

Array Bounds



- All array subscripts begin at **0**:

```
int list[] = new int [10];
for (int i = 0; i < list.length; i++) {
     System.out.println(list[i]);
}
```

Array Resizing



- Cannot **resize** an array
- Can use the same reference variable to refer to an entirely new array:

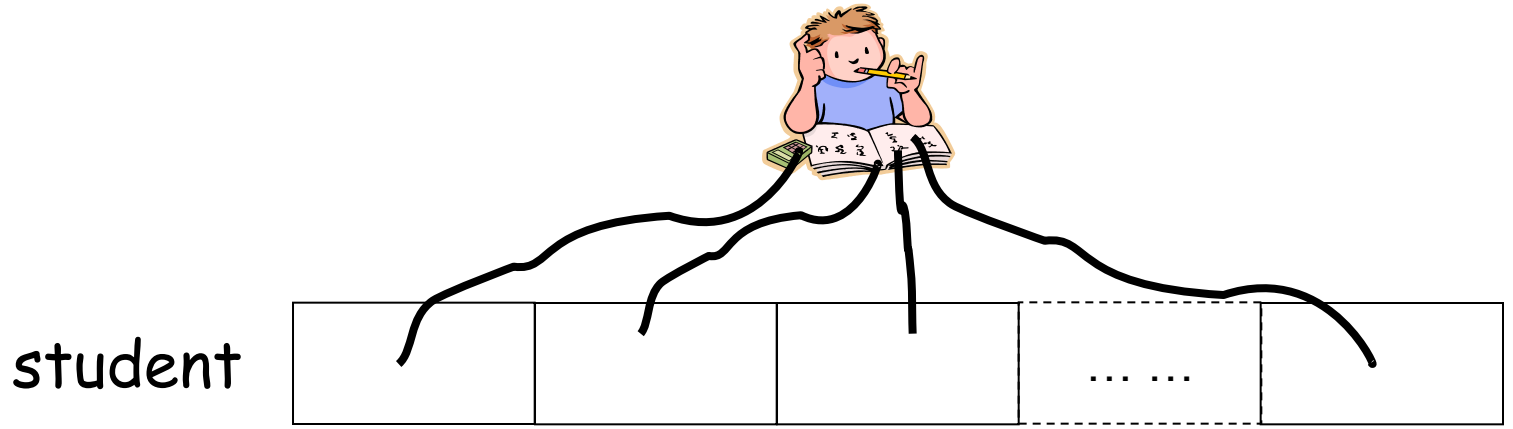
```
int elements[] = new int[6];
elements = new int[10];
```

Copying Arrays

```
1 //original array
2 int elements[] = { 1, 2, 3, 4, 5, 6 };
3
4 // new larger array
5 int hold[] = { 10, 9, 8, 7, 6, 5, 4, 3, 2,
                  1 };
6
7 for(int i = 0; i < elements.length; i++){
8         elements[i] = hold[i];
9 }
```

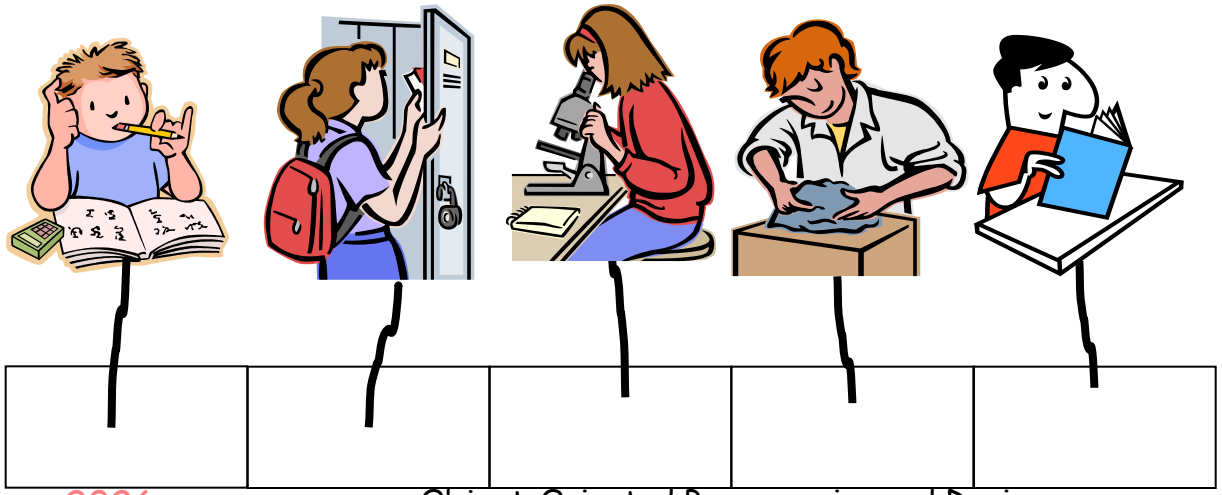
Manipulating Arrays of Objects

```
1 Student[] student = new Student[20];
2 Student s = new Student("Fred");
3 for(int i = 0; i < student.length; i++){
4     student[i] = s;
5}
```



Manipulating Arrays of Objects

```
1 Student[] student = new Student[5];
2 student[0] = new Student("Fred");
3 student[1] = new Student("Mary");
4 student[2] = new Student("Alice");
5 student[3] = new Student("Joe");
6 student[4] = new Student("Michle");
```



ArrayList



- The class java.util.ArrayList<E> implements a collection of objects that can grow to accommodate new items when the collection is full.
  - ArrayList(). Constructs an empty collection.
  - int size(). Returns the number of objects in the collection.
  - boolean isEmpty(). Determines if there are no objects in the collection.
  - boolean contains(Object elem). Determines if the specified object is an element of the collection (as determined by the method equals).
  - boolean add(E o). Appends the specified object to the end of the collection.
  - void add(int index, E element). Inserts the specified object at the specified index position, shifting any subsequent elements to the right (adds one to their indices).

ArrayList



- The class java.util.ArrayList<E> implements a collection of objects that can grow to accommodate new items when the collection is full.
  - E get(int index). Returns the object at the specified position.
  - public E set(int index, E element). Replaces the element at the specified index position with the specified object.
  - public boolean remove(Object o). Removes the first occurrence of the specified object (using method equals), shifting any subsequent elements to the left (subtracts one from their indices).
  - E remove(int index). Returns the object at the specified position after first removing it from the collection and shifting any subsequent elements to the left (subtracts one from their indices).

ArrayList

```
ArrayList<Employee> employees = new
  ArrayList<Employee>();


employees.add(new Employee("John Smith"));
employees.add(new Employee("Mary Williams"));
employees.add(new Employee("Peter
  Jefferson"));


Employee firstEmployee = employees.get(0);
Employee secondEmployee = employees.get(1);
Employee thirdEmployee = employees.get(2);
```

ArrayList

```
ArrayList<Integer> numbers = new
  ArrayList<Integer>();


numbers.add(1);

int num = numbers.get(0);
```

Iterator



- An ***iterator*** is an object for traversing a collection from start to finish.
- The class java.util.Iterator<E> provides the following methods:
  - *boolean hasNext()*. Returns true if the iteration has more elements.
  - *E next()*. Returns the next element in the iteration.
  - *void remove()*. Removes from the collection the last element returned by the iterator.

Iterator

```
ArrayList<String> list = new Arralist<String>();

list.add("ArrayList");
list.add(" and ");
list.add("Iterators");

String result = "";

for (Iterator<String> listIterator = list.iterator();
                    listIterator.hasNext(); ) {
      result += listIterator.next();
}
stdout.println(result);
```

Using the For-Each Loop in Collections

```
ArrayList<String> list = new Arralist<String>();

list.add("ArrayList");
list.add(" and ");
list.add("for-each");

String result = "";

for (String element : list) {
      result += element;
}

stdout.println(result);
```

For-Each Loop



- The for-each loop provides a simple way to iterate through the elements of a collection. In the for-each loop, you should specify the collection to iterate through, and a variable to access each element.

For-Each Loop



- The for-each loop can be used only on those classes that implement the interface java.util.Iterable<T>. The class must include an implements clause in the class declaration, and declare the method iterator.

```
public class MyCollection implements
Iterable<T>  {   ...

   public Iterator<T> iterator() {

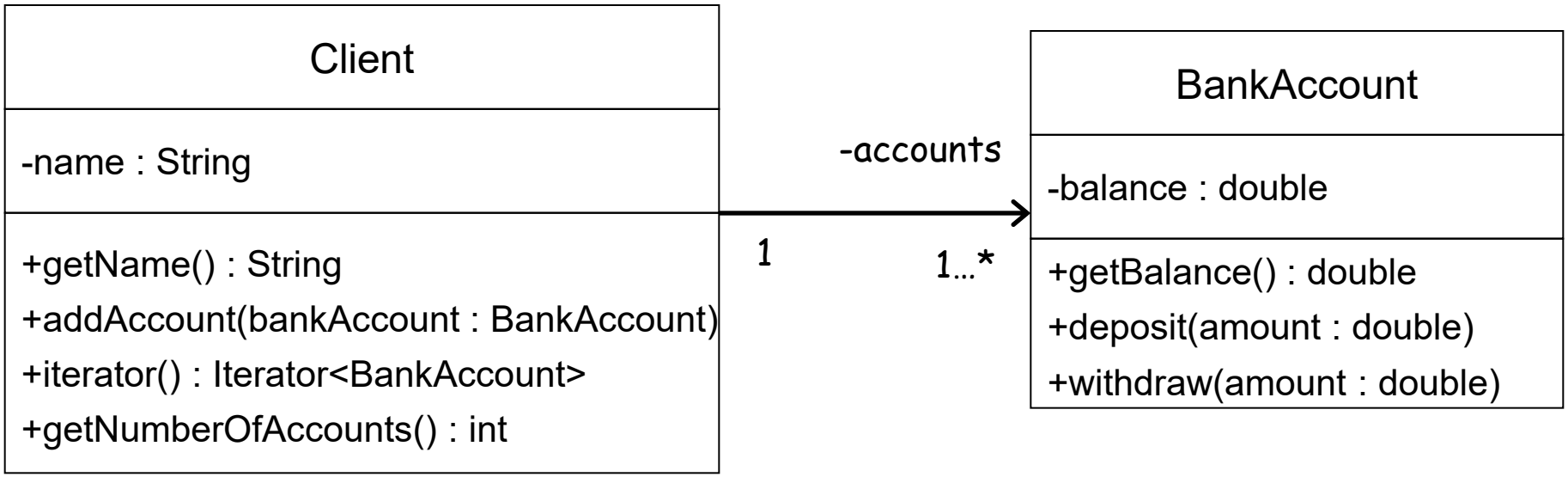
         ...

   }

}
```

For example




Client
-name : String
+getName() : String
+addAccount(bankAccount : BankAccount)
+iterator() : Iterator<BankAccount>
+getNumberOfAccounts() : int
-accounts
1
1...*
BankAccount
-balance : double
+getBalance() : double
+deposit(amount : double)
+withdraw(amount : double)

BankAccount

```
public class BankAccount {
   private double balance;
   public BankAccount() {
       this.balance = 0.0;
   }
   public double getBalance() {
       return this.balance;
   }
```

BankAccount

```
    public boolean deposit(double amount) {
    if (amount > 0) {
          this.balance += amount;
          return true;
    } else {
          return false;
    }
}
```

BankAccount

```
    public boolean withdraw(double amount) {
        if (amount > 0 && this.balance >= amount) {
            this.balance -= amount;
            return true;
        } else {
            return false;
        }
    }
}
```

Client

```
public class Client  implements
  Iterable<BankAccount>  {


  /* Name of client */

  private String name;


  /* Collection of <code>BankAccounts</code>
  objects.*/

  private ArrayList<BankAccount>  accounts;
```

Client

```
public Client(String initialName)  {
      this.name = initialName;
      this.accounts = new ArrayList<BankAccount>();
   }

public String  getName() {
      return this.name;
   }

public void  addAccount(BankAccount bankAccount)  {
      this.accounts.add(bankAccount);
   }
```

Client

```
public Iterator<BankAccount>  iterator() {
      return this.accounts.iterator();
   }

public int  getNumberOfAccounts()   {
      return this.accounts.size();
   }
}
```

```
public class TestClient  {
   public static void main(String[] args) {
        BankAccount accountOne = new BankAccount();
        BankAccount accountTwo = new BankAccount();
        BankAccount accountThree = new BankAccount();
        accountOne.deposit(1000.0);
        accountTwo.deposit(2000.0);
        accountThree.deposit(3000.0);
        Client client= new Client("John Smith");
        client.addAccount(accountOne);
        client.addAccount(accountTwo);
        client.addAccount(accountThree);
        double totalBalance = 0.0;
        for (BankAccount account : client) {
                totalBalance += account.getBalance();
        }
        if (totalBalance != 6000.0) {
                System.out.println("** Test failure");
        }
        System.out.println("done");   }}
```

4.5 Inheritance

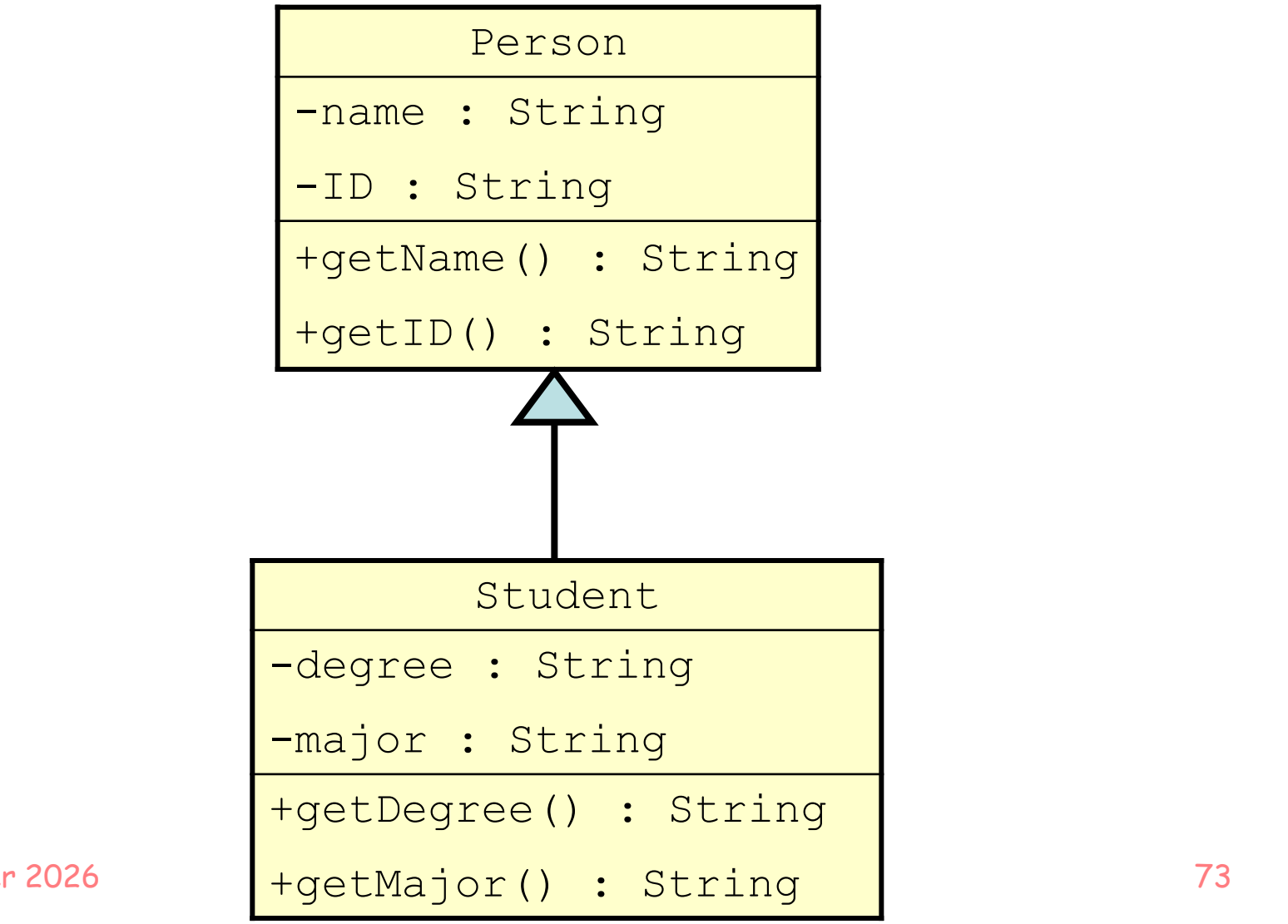


Inheritance



- **inheritance** is a mechanism for defining a new class by stating only the differences (in terms of features) between the new class and another class that we've already established.

Inheritance



Inheritance

```
public class Person {
   private String  name;
   private String  ID;
   public Person (String initialName, String initialID) {
        name = initialName;
        ID = initialID;
   }
   public String getName() {
        return this.name;
   }
   public String getID() {
        return this.ID;
   }
}
```

Inheritance

```
public class Student extends Person {
   private String degree;
   private String major;

   public Student(String initialName, String initialID,
                  String initialDegree, String initialMajor) {

        super(initialName, initialID);
       degree = initialDegree;
       major = initialMajor;
   }
   public String getDegree() {
       return this.degree;
   }

   public String getMajor() {
       return this.major;
   }
}
```

Inheritance

```
public class SRS {

   public void static main(String args[]) {

        Person person = new Person("Alice", "20100119880112206x");
        Student john = new Student("John", "20100119780112113x",
                                   "bachelor", "SE");

   System.out.println("The person's name is " + person.getName());
   System.out.println("The person's ID is " + person.getID());
   System.out.println();

System.out.println("The student's name is " + john.getName());
System.out.println("The student's ID is " + john.getID());
System.out.println("The student's degree is " + john.getDegree());
System.out.println("The student's major is " + john.getMajor());
   }
}
```

Inheritance

Output:

```
The person's name is Alice
The person's ID is 20100119880112206x

The student's name is John
The student's ID is 20100119780112113x
The student's degree is bachelor
The student's major is SE
```

Inheritance

```
public class SRS {

   public void static main(String args[]) {

        Person person = new Person("Alice", "20100119880112206x");
        Student john = new Student("John", "20100119780112113x",
                                   "bachelor", "SE");

        person = john;

   System.out.println("The person's name is " + person.getName());
   System.out.println("The person's ID is " + person.getID());
   System.out.println();

System.out.println("The student's name is " + john.getName());
System.out.println("The student's ID is " + john.getID());
System.out.println("The student's degree is " + john.getDegree());
System.out.println("The student's major is " + john.getMajor());
   }
}
```

Inheritance

Output:

```
The person's name is John
The person's ID is 20100119780112113x

The student's name is John
The student's ID is 20100119780112113x
The student's degree is bachelor
The student's major is SE
```

Inheritance

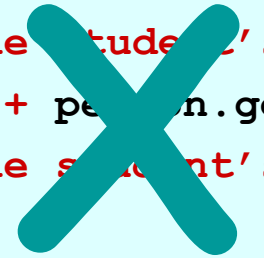
```
public class SRS {

   public void static main(String args[]) {

        Person person = new Person("Alice", "20100119880112206x");
        Student john = new Student("John", "20100119780112113x",
                                   "bachelor", "SE");

        person = john;

System.out.println("The student's name is " + person.getName());
System.out.println("The student's ID is " + person.getID());
System.out.println("The student's degree is "
                   + person.getDegree());
System.out.println("The student's major is " + person.getMajor());
   }
}
```



Casting Object

```
public class SRS {

   public void static main(String args[]) {

        Person person = new Person("Alice", "20100119880112206x");
        Student john = new Student("John", "20100119780112113x",
                                   "bachelor", "SE");

        person = john;

System.out.println("The student's name is " + person.getName());
System.out.println("The student's ID is " + person.getID());
System.out.println("The student's degree is "
                     + ((Student)person).getDegree());
System.out.println("The student's major is "
                     + ((Student)person). getMajor());
   }
}
```

Casting Object

Output:

```
The student's name is John
The student's ID is 201001197801121l3x
The student's degree is bachelor
The student's major is SE
```

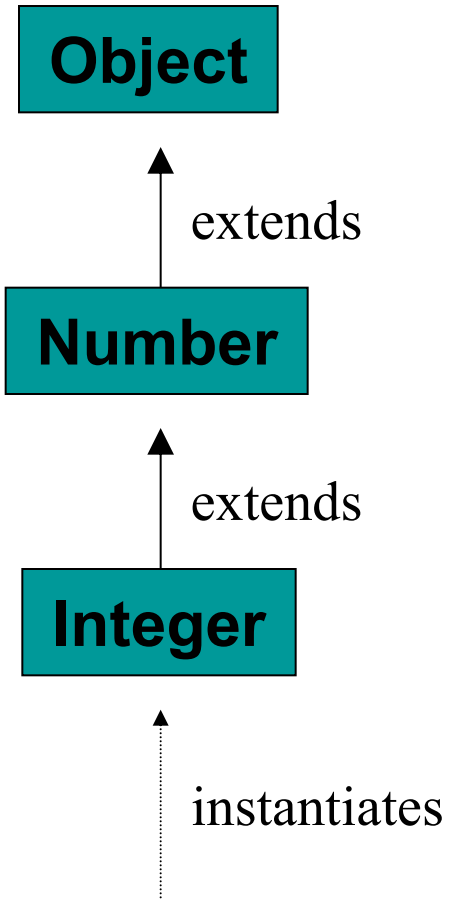
Inheritance



- When we take advantage of inheritance, the original class that we're starting from— person, in this case—is called the (**direct**) **superclass.** The new class—student—is called a (**direct**) **subclass.** A subclass is said to **extend** its direct superclass.

Class Hierarchies

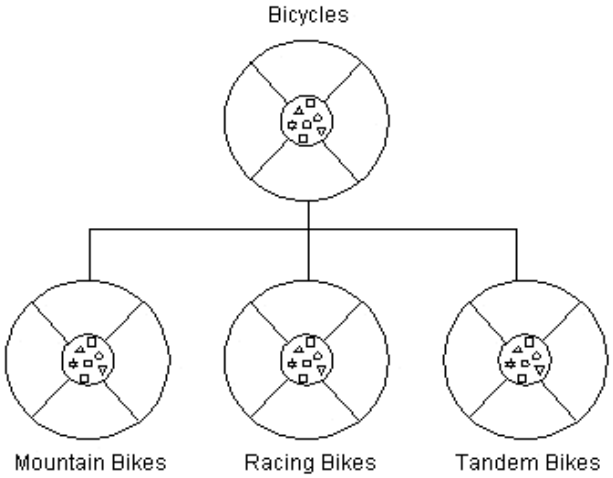
- Every object belongs to a class
  - The objects that are members of a class are its instances
- Every class (except **Object**) has a *superclass*
  - In Java, **Object** is the root of the entire class hierarchy
- When defining new classes, the developer must decide which class is the appropriate *superclass*




```
Integer zero = new Integer(0);
```

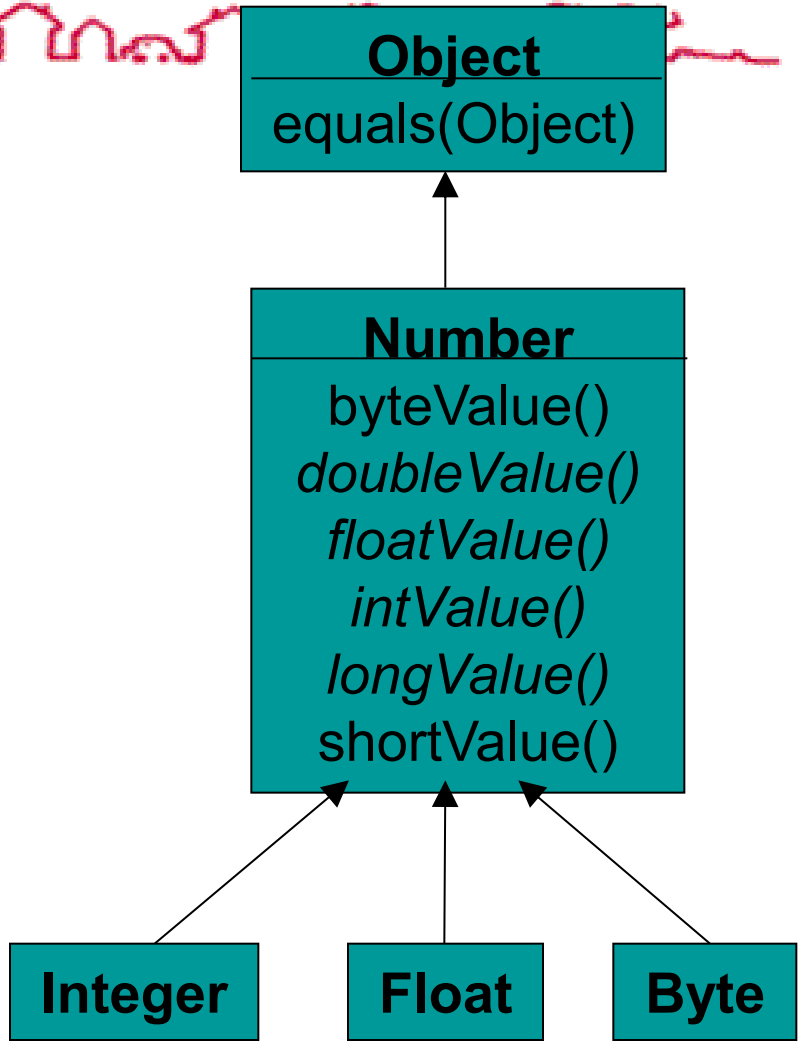
Class Hierarchy Example

- Object-oriented systems allow classes to be defined in terms of other classes
- Mountain bikes, racing bikes, and tandems are all different kinds of bicycles, so they are all *subclasses* of the bicycle class
- The *superclass* is the bicycle class
- **Subclasses inherit variables and methods from the superclass.**



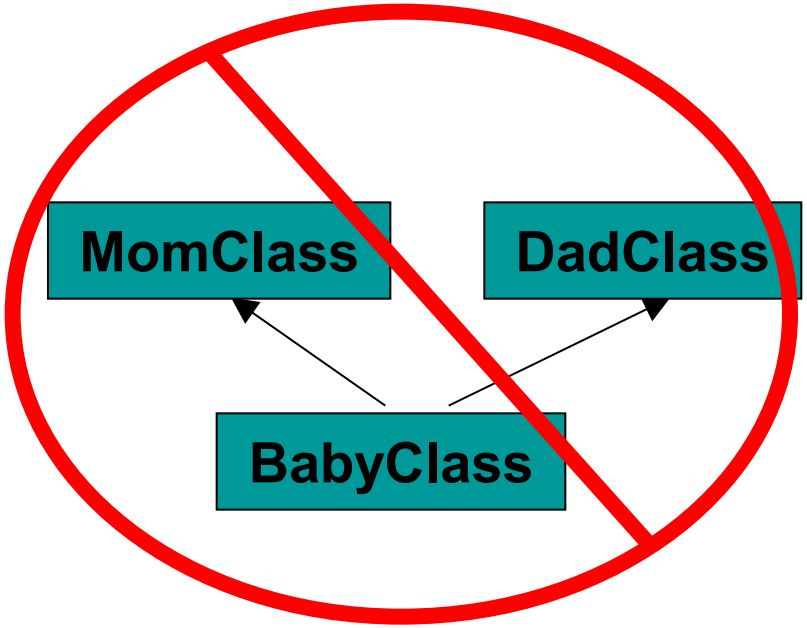
Specialization and Generalization

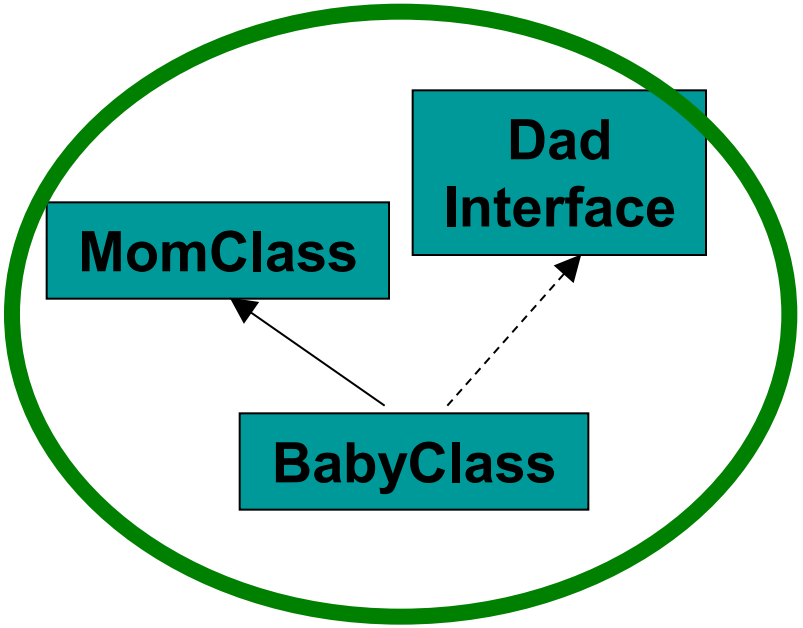
- A *subclass* is a specialization of its superclass
  - Specialize the state and behavior by adding fields and extending or changing methods
- A superclass is a generalization of its subclasses
  - Common state and behavior can be moved to the superclass where it is available to all subclasses
  - Only needs to be written once



Multiple Inheritance

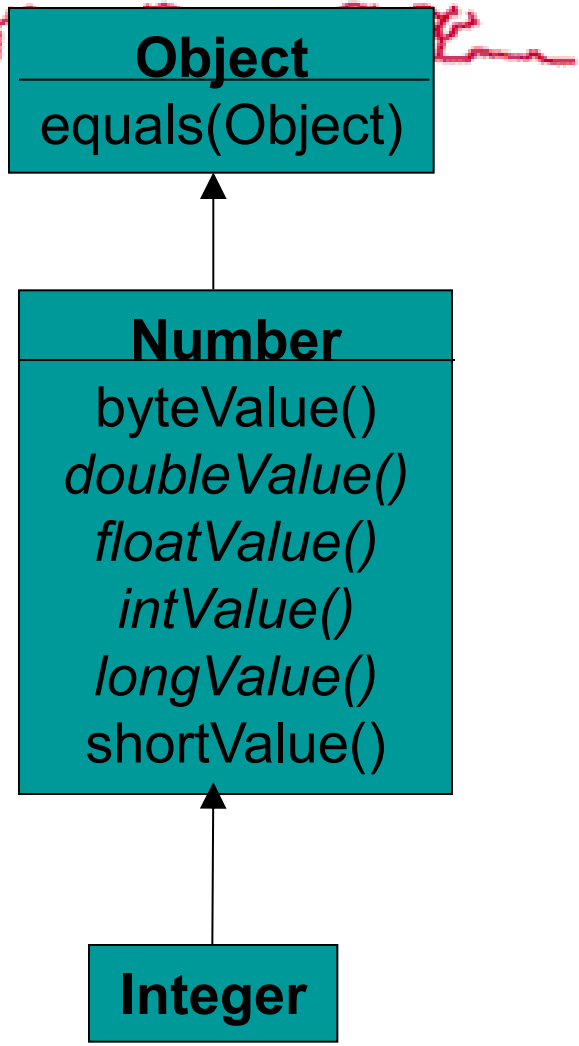
- Java does not support multiple inheritance
  - Every Java class except Object has exactly one immediate *superclass* (Object does not have a superclass)
- You can force classes that are not related by inheritance to implement a common set of methods using **interfaces** (this will be discussed in a later section)





Inheriting Fields and Methods

- Each subclass inherits the fields of its superclass
  - These fields may have been inherited from classes further up in the hierarchy
- Each subclass inherits the methods of its superclass
  - An object will understand all messages its class or superclass has inherited or implemented




```
Integer zero = new Integer(0);
    if (zero.equals(x)) {
   byte b = zero.byteValue();
                ...
               }
```

Access Modifiers(1)



- Variables and methods in Java have access restrictions, described by the following access modifiers:
  - private:
    - Access is limited to the class in which the member is declared
    - Example: `private int x;`
  - default (this means no modifier is used):
    - Access is limited to the package in which the member is declared
    - Example: `int x;`

Access Modifiers(2)



- Variables and methods in Java have access restrictions, described by the following access modifiers:
  - protected:
    - Access is limited to the package in which the member is declared, as well as all subclasses of its class
    - Example: `protected void setName() { . . . }`
  - public:
    - The member is accessible to all classes in all packages
    - Example: `public String getName() { . . . }`

Inheritance

- A superclass object can reference a subclass object;
  - person = john;
- A subclass object can not reference a superclass object;
  - john = person; 

Inheritance

- A superclass object can reference a subclass object;
  - person = john;
- The reference person, which points to an Student object, ***cannot*** be used to invoke the Student methods!
  - String degree = person.getDegree( ); ✗
  - String name = person.getName( ); √

Casting Object

- A superclass object can reference a subclass object;
  - person = john;
- The reference person, which points to an Student object, *cannot* be used to invoke the Student methods!
  - String degree = ((**Student**)person).getDegree( );
- If person references a Person object, then cannot be casting.
  - Person person = new Person("John", "**`20100119780112113x`**");
  - String degree = ((Student)person).getDegree( ); 

instance of



- The `instanceof` operator:

```
object instanceof ClassX
```

  - An `instanceof` expression evaluates to `true` if the specified object is an instance of the specified class.
  - The expression will also evaluate to `true` if `object` is an instance of a `ClassX` subclass.
  - If the left-hand operand is `null`, instanceof expression evaluates to `false`.

instance of & casting

- The `instanceof` operator is often used to avoid an illegal cast :

```
public class SRS {
   public void static main(String args[]) {
        Person person = new Student("John", "20100119780112113x",
                                    "bachelor", "SE");
        if(person instance of Student){
System.out.println("The student's name is " + person.getName());
System.out.println("The student's ID is " + person.getID());
System.out.println("The student's degree is "
                    + ((Student)person).getDegree());
System.out.println("The student's major is "
                    + ((Student)person). getMajor());
        }
   }
}
```

The “is a” Nature of Inheritance



- Inheritance is often referred to as the “is a” relationship between two classes, because if a class B is derived from a class A, then B truly *is a* special case of A.
- Subclasses are special cases of their superclasses, the term specialization is used to refer to the process of deriving one class from another.
- Generalization, on the other hand, is a term used to refer to recognizing the common features of several existing classes and creating a new, common superclass for them all.

Composition vs. Inheritance



- Inheritance
  - "Is a" relationship
  - **`Teacher`** *is an* **`Employee`**
- Composition
  - "Has a" relationship
  - **`Employee`** *has a* **`TelephoneNumber`**

The Benefits of Inheritance



- We dramatically reduce code redundancy
- Subclasses are much more succinct than they would be without inheritance.
- Through inheritance, we can reuse and extend code that has already been thoroughly tested without modifying it.
- Best of all, we can derive a new class from an existing class even if we don't own the source code for the latter!

4.6 Overriding



Overriding



- Overriding involves "rewiring" how a method works internally, without changing the client code interface to/signature of that method.

Overriding

- For example,
  - Superclass **`Shape`**
  - Subclasses **`Circle`**, **`Rectangle`** and **`Square`**
  - Each class draws itself according to type of class
    - **`Shape`** has method **`calculateArea`**
    - Each subclass overrides method **`calculateArea`**

Overriding

```
public class Shape {
  public double calculateArea(){
     System.out.println("This is in Shape
  class.");
     return 0;
  }
}
public class Circle extends Shape {
  private double radius;
  public double calculateArea(){
     return 3.14*radius*radius;
  }}
```

Overriding

```
public class Rectangle extends Shape {
   private double length;
   private double width;
   public double calculateArea(){
       return length*width;
   }
}
public class Square extends Shape {
   private double length;
   public double calculateArea(){
       return length*length;
   }
}
```

Overriding

```
public class TestShape {
   public static void main(String args[]){
        Shape shape;
        shape = new Circle(2);
        System.out.println("The shape's area is "
              + shape.calculateArea());
        shape = new Rectangle(3, 4);
        System.out.println("The shape's area is "
              + shape. calculateArea());
        shape = new Square(3);
        System.out.println("The shape's area is "
              + shape. calculateArea());
   }
}
```

4.7 “super” keyword



Inheritance and Constructors



- Only constructors within the class being instantiated and within the immediate superclass can be invoked
- Constructors in the superclass are invoked with the keyword `super` and the parameter list
  - The parameter list must match that of an existing constructor in the superclass
- Constructors in the same class are invoked with the keyword `this` and the parameter list
- The first line of your constructor can be one of:
  - `super(…);`
  - `this(…);`

The Superclass in Object Construction



- Superclass objects are built before the subclass
  - The compiler supplies an implicit `super()` call for all constructors
- `super(...)` initializes superclass members
- If the first line of your constructor is not a call to another constructor, `super()` is called automatically
  - Zero argument constructor in the superclass is called as a result
  - This can cause an error if the superclass does not have a zero-argument constructor

Default Constructors



- If you do not provide any constructors, a default zero-argument constructor is provided for you
- If you implement any constructor, Java will no longer provide you with the default zero-argument constructor
  - You can write your own zero-argument constructor which behaves like the default constructor (makes an implicit call to `super()`)

Constructors Are Not Inherited

```
public class Person {
   private String  name;
   private String  ID;
   public Person (String initialName, String initialID) {
        name = initialName;
        ID = initialID;
   }
   public String getName() {
        return this.name;
   }
   public String getID() {
        return this.ID;
   }
}
```

Constructors Are Not Inherited

```
public class Student extends Person {
   private String degree;
   private String major;

   public Student(String initialName, String initialID,
                  String initialDegree, String initialMajor) {

        super(initialName, initialID);
       degree = initialDegree;
       major = initialMajor;
   }
   public String getDegree() {
       return this.degree;
   }

   public String getMajor() {
       return this.major;
   }
}
```

“super” keyword

```
public class Person {
   private String  name;
   private String  ID;
   public Person (String initialName, String initialID) {
        name = initialName;
        ID = initialID;
   }
   public String getName() {
         return this.name;
   }
   public String getID() {
         return this.ID;
   }
   public String toString() {
         return this.ID + "_" + this.Name;
   }
}
```

“super” keyword

```
public class Student extends Person {
    private String degree;
    private String major;

    public Student(String initialName, String initialID,
                   String initialDegree, String initialMajor) {

         super(initialName, initialID);
         degree = initialDegree;
         major = initialMajor;
    }
    public String getDegree() {
         return this.degree;
    }

    public String getMajor() {
         return this.major;
    }
    public String toString() {
         return super.toString() + "_" + this.degree + "_" + this.major;
    }
}
```

More on `this` and `super`



- `this` and `super` can be used in instance methods and constructors in Java
  - They cannot be used in class (static) methods
- The keyword `super` lets you use code from a superclass when you are overriding a method
- `this` and `super` affect method look-up
  - `this`: method look-up starts in the current class
  - `super`: method look-up starts in the immediate superclass

4.8 equals() and toString() Method



equals() method

- In Java, all classes descend, directly or indirectly, from class **Object** , so all classes inherit the methods defined in class Object. Now we will use two of these methods, ***equals*** and ***toString***, extensively .
- The method ***equals*** compares two objects and returns true if and only if they are equal.
  - `boolean equals(Object obj)`

equals() method

```
Point pointOne = new Point(10, 100);
Point pointTwo = new Point(10, 100);

Point pointThree = pointOne;

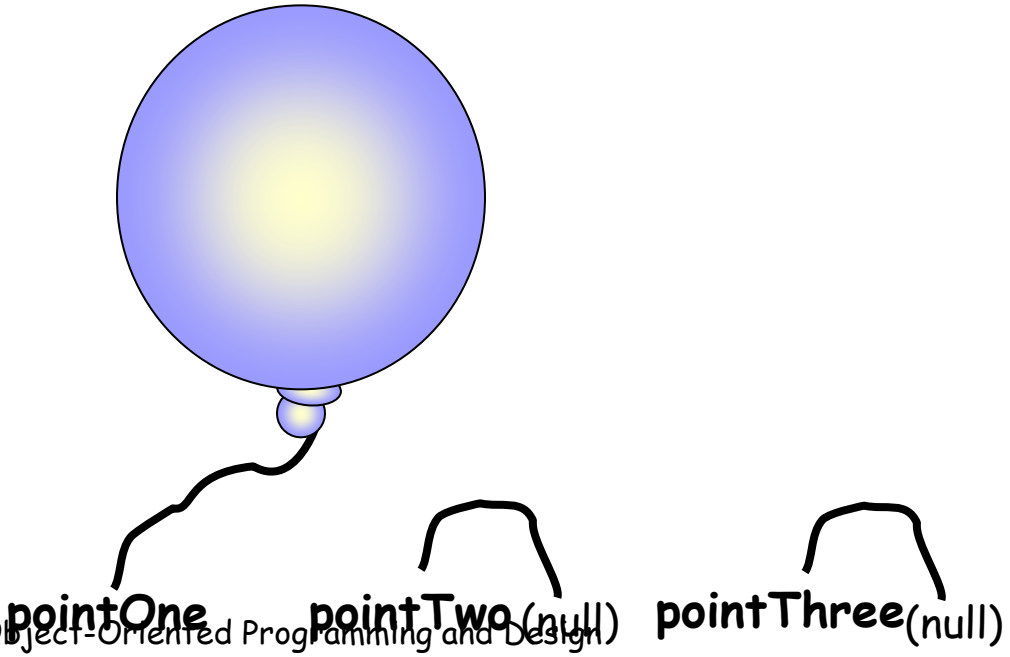
if (pointOne.equals(pointTwo)) {
        System.out.println("pointOne and pointTwo are equal");
        } else {
        System.out.println("pointOne and pointTwo are different");
}

if (pointOne.equals(pointThree)) {
        System.out.println("pointOne and pointThree are equal");
        } else {
        System.out.println("pointOne and pointThree are different");
}
```

equals() method

```
Point pointOne = new Point(10, 100);
Point pointTwo = new Point(10, 100);

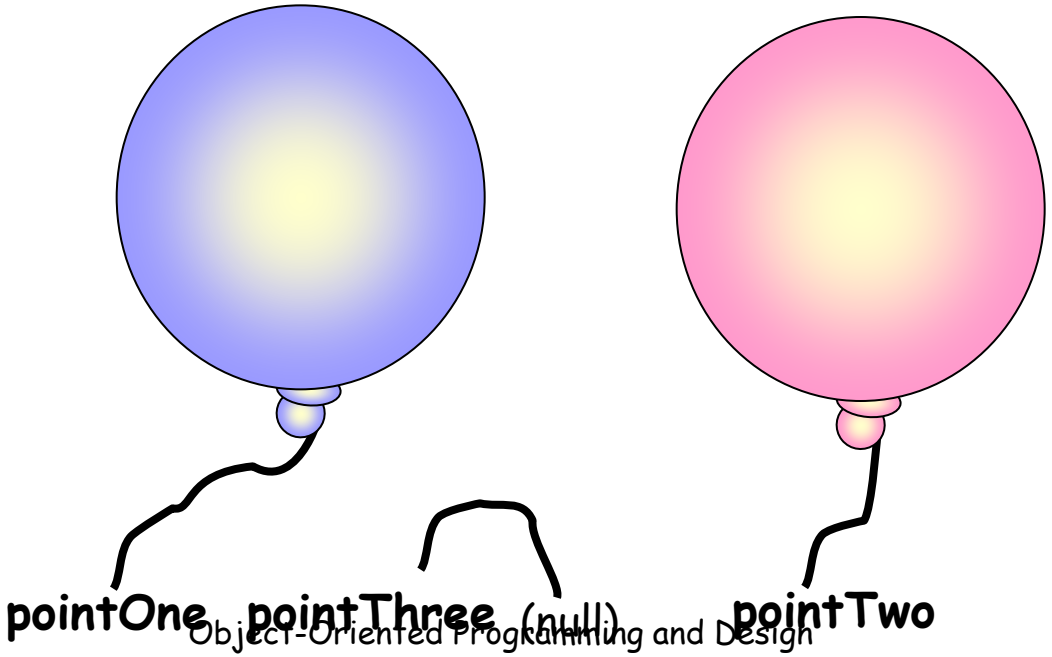
Point pointThree = pointOne;
```



equals() method

```
Point pointOne = new Point(10, 100);
Point pointTwo = new Point(10, 100);

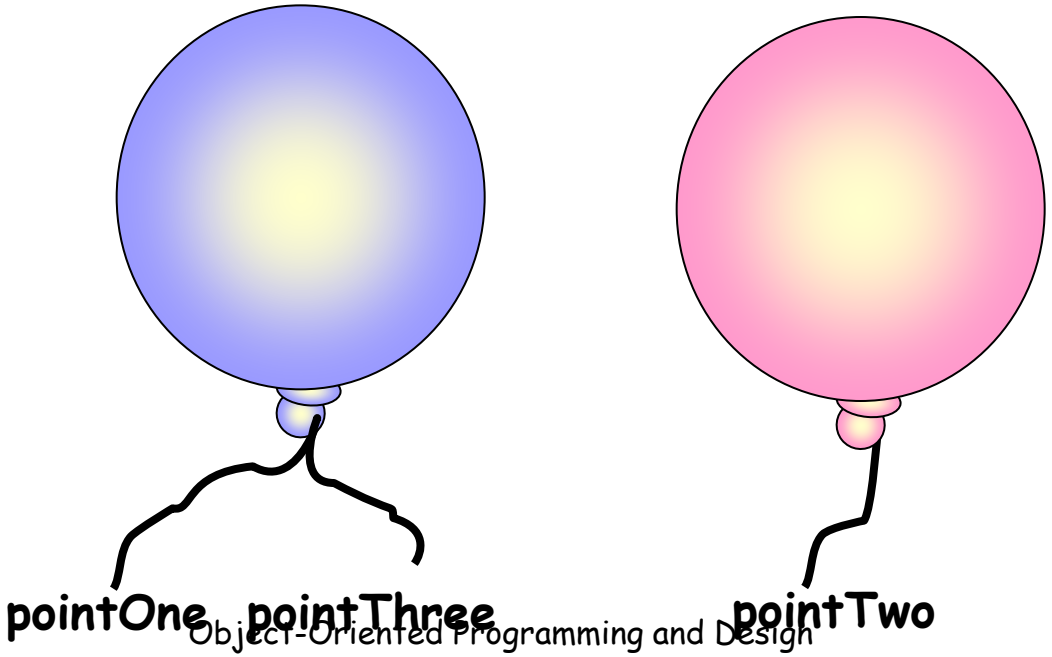
Point pointThree = pointOne;
```



equals() method

```
Point pointOne = new Point(10, 100);
Point pointTwo = new Point(10, 100);

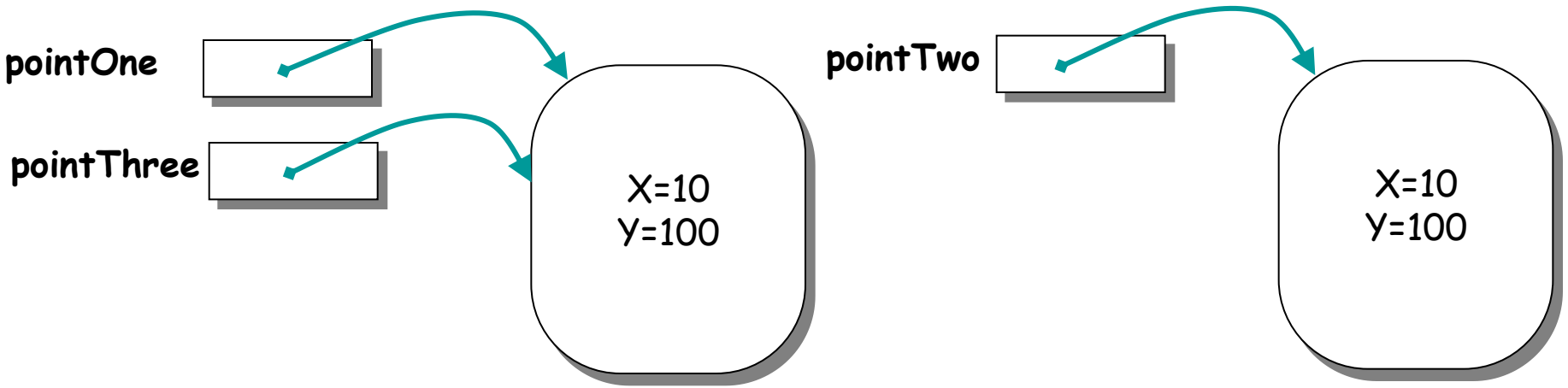
Point pointThree = pointOne;
```



equals() method

```
Point pointOne = new Point(10, 100);
Point pointTwo = new Point(10, 100);

Point pointThree = pointOne;
```



Override equals() method



- The method ***equals***, as defined in **Object**, is not appropriate for most classes, so many of them override it.
- The version of ***equals*** in most classes returns true when the objects being compared have the same state, that is, contain the same ***data***. For example, class Point could define the following version of equals:

Override equals() method

```
public boolean  equals(Object  object)  {
        if (object instanceof Point) {
           Point point = (Point) object;
           return point.getX() == getX() &&
                        point.getY() == getY();
        } else {
           return false;       }    }
```

Override equals() method

```
Point pointOne = new Point(10, 100);
Point pointTwo = new Point(10, 100);
Point pointThree = new Point(50, 500);

if (pointOne.equals(pointTwo)) {
     System.out.println("pointOne and pointTwo are equal");
} else {
   System.out.println("pointOne and pointTwo are different");
}

if (pointOne.equals(pointThree)) {
     System.out.println("pointOne and pointThree are equal");
} else {
   System.out.println("pointOne and pointThree are different");
}
```

toString() method

- Class Object defines a method called ***toString*** that returns the string representation of the invoking object. The version of this method defined in class Object returns a String with the following format:

  ClassName@number

toString() method

- It is recommended that all subclasses override method ***toString***. Classes typically redefine ***toString*** so that the String returned contains textual representation of the object. For example, consider the following version of ***toString*** in class Point:

```
public String  toString()  {
      return  "(" + getX() + "," + getY() + ")";
  }
```

toString() method

```
Point pointOne = new Point(10, 100);
Point pointTwo = new Point(-20, 200);
Point pointThree = new Point(50, -500);
System.out.println(pointOne);
System.out.println(pointTwo);
System.out.println(pointThree);
System.out.println();
```

```
output:
(10,100)
(-20,200)
(50,-500)
```

Summary

- Association and link
- Multiplicity
- Aggregation and composition
- Collection, ArrayList&Iterator
- Inheritance
- Overriding
- Super
- equals() and toString()

Excecises

- Read textbook, chapter 5 & chapter 6.